Doc. no.	P2139R1
Date:	2020-06-14
Project:	Programming Language C++
Audience:	Evolution Working Group Incubator
	Library Evolution Working Group Incubator
Reply to:	Alisdair Meredith <ameredith1@bloomberg.net>

Reviewing Deprecated Facilities of C++20 for C++23

Table of Contents

Revision History

Revision 1

Revision of the paper for the 2020 June mailing, following early feedback.

- Clean up document format using a html validator
- Note which compiler versions start issuing warnings
- Record EWGI review of subclauses D.1 - D.4.
- Start collecting integrated wording for eventual core review, where EWGI have provided a recommendation.
- Note that users can work around deprecation in D.1 with a simple unary +
- Update wording for EWGI preference to remove just floating point interaction in D.1
- Propose Annex C wording for D.1
- Completely revised recommendations for D.3 (comma expressions in array bounds), guided by feedback on R0 of this paper.
- Propose Annex C wording for D.3 following EWGI recommendation to remove.
- Propose Annex C wording for D.4 following EWGI recommendation to remove.
- A more fine-grained analysis of D.5 (volatile operations), as this deprecation is really four cases in one clause.

- Initial drafting of core wording to remove D.5, if we so choose.
- Note that no currently available compiler warns on the deprecation of D.6 (Redeclare `constexpr` members)
- Example of an iterator implementation idiom that requires slightly more work to resolve D.8 deprecation.
- Note that Clang 10 also now warns on the deprecations in D.8.
- Added a note to D.9 that we expect compilers will never give a deprecation warning on the use of these headers
- For D.10 (library *Requires* paragraphs), revised suggestion of *Mandates* to *Preconditions* where both run-time and compile-time requirements applied.

Original

Original version of the paper for the 2020 post-Prague mailing.

1 Abstract

This paper evaluates all the existing deprecated facilities in the C++20 standard, and recommends removing a subset from Annex D in C++23, either by removing from the standard entirely, or by undeprecating and restoring to the main text.

2 Stating the problem

With the release of a new C++ standard, we get an opportunity to revisit the features identified for deprecation, and consider if we are prepared to clear any out yet, either by removing completely from the standard, or by reversing the deprecation decision and restoring the feature to full service. This paper makes no attempt to offer proposals for removing features other than those deprecated in Annex D, nor does it attempt to identify new candidates for deprecation.

In an ideal world, the start of every release cycle would cleanse the list of deprecated features entirely, allowing the language and library to evolve cleanly without holding too much deadweight. This paper is an attempt to consolidate all likely requests (and papers) to review isolated features on Annex D, and gain efficiency by performing one consolidated review. In practice, C++ has some long-term deprecated facilities that are difficult to remove, and equally difficult to rehabilitate. It seems reasonable that work could be split into follow-up papers if a direction has appeal, but needs more work for consensus to advance; one possible goal of this paper is to discover which features we may be interested in following up while maintaining the tempo of an efficient high level review.

The benefits of making the choice to remove features early in a standard cycle is that we will get the most experience we can from the bleeding-edge adopters whether a particular removal is more problematic than expected - but even this data point is limited, as bleeding-edge adopters typically have less reliance on deprecated features, eagerly adopting the newer replacement facilities.

However, do note that with the three year release cadence for the C++ standard, we will often be re-evaluating features whose deprecated status has barely reached print.

We have precedent that Core language features are good targets for removal, typically taking two standard cycles to remove a deprecated feature, although often prepared to entirely remove a feature even without a period of deprecation, if the cause is strong enough.

The library experience has been mixed, with no desire to remove anything without a period of deprecation (other than `gets`) and no precedent prior to C++17 for actually removing deprecated features.

The precedent set for the library in C++17 seems to be keener to clear out the old code more quickly though, even removing features deprecated as recently as the previous standard, with the ink still drying on the text! Accordingly, this paper will be fairly aggressive in its attempt to clear out old libraries. This is perhaps more reasonable for the library clauses than the core language, as the Zombie Names clause, **16.5.4.3.1 [zombie.names]** allows vendors to continue shipping features long after they have left the standard, as long as their existing customers rely on them.

3 Proposed Solution

We will review each deprecated facility, and make a strong and a weak recommendation. The strong recommendation is the preferred direction of the authors, who lean towards early removal. There will also be a weak recommendation, which is an alternative proposal for the evolution groups to consider, if the strong recommendation does not find favor. Finally, wording is generally provided for both the strong and weak recommendations, which will be collated into a unified Proposed Wording section once the various recommendations have been given direction by the corresponding evolution group.

All proposed wording is relative to N4861, which best corresponds to the C++20 DIS.

The intent is that the incubator groups review these proposals by telecon throughout 2020, indicating which features should be given more attention to gain implementation experience in time to inform the evolution groups when face-to-face meetings resume in (hopefully) 2021. Recommendations will be tracked in the table below.

Checklist for recommendations:

Subclause	Introduced	Deprecated	Paper	Feature	Strong Recommendation	Weak Recommendation	Incubator Recommendation	Final Decision
D.1	C++98	C++20	P1120R0	Arithmetic conversion on enumerations	No action	Remove now	Partial removal	pending...
D.2	C++11	C++20	P0806R2	Implicit capture of `*this` by reference	Remove now	No action	No action	pending...
D.3	C++98	C++20	P1161R3	Comma operator in subscript expressions	P2128R1	Remove now	Remove now	pending...
D.4	C++98	C++20	P1120R0	Array comparisons	Remove now	No action	Remove now	pending...
D.5	C++98	C++20	P1152R4	Deprecated use of `volatile`	No action	Remove now	pending...	pending...
D.6	C++11	C++17	P0386R2	Reclare `constexpr` members	Remove now	No action	pending...	pending...
D.7	C++98	C++20	P1815R2	Non-local use of TU-local entities	No action	No action yet	pending...	pending...
D.8	C++98	C++11	N3203	Implicit special members	Remove on copy Undeprecate on dtor	No action yet	pending...	pending...
D.9	C++98	C++98		C `<*.h>` headers	Undeprecate	Remove now	pending...	pending...
D.10	C++98	C++20		*Requires:* clauses	Remove now	No action	pending...	pending...
D.11	C++98	C++20	P0768R1	`relops`	Remove now	Fix mandates	pending...	pending...
D.12	C++98	C++98		`char *` streams	No action	No action yet	pending...	pending...
D.13	C++11	C++20	P0767R1	Deprecated type traits	Fix mandates	Remove now	pending...	pending...
D.14	C++11	C++20	P1831R1	`volatile tuple` API	No action	No action	pending...	pending...
D.15	C++17	C++20	P1831R1	`volatile variant` API	Remove now	No action	pending...	pending...
D.16	C++98	C++17	P0174R2	`std::iterator`	Remove now	Undeprecate	pending...	pending...
D.17	C++11	C++20	P1252R2	`move_iterator::operator->`	No action	Remove now	pending...	pending...
D.18	C++11	C++20	P0718R2	C API to use `shared_ptr` atomically	Remove now	Fix mandates	pending...	pending...
D.19	C++98	C++20	P0966R1	`basic_string::reserve()`	No action	Remove now	pending...	pending...
D.20	C++11	C++17	P0618R0	`<codecvt>`	Remove now	No action yet	pending...	pending...
D.21	C++11	C++17	P0618R0	`wstring_convert` et al.	Remove now	Fix mandates	pending...	pending...
D.22	C++11	C++20	P0482R6	Deprecated locale category facets	No action	No action	pending...	pending...
D.23	C++17	C++20	P0482R6	`filesystem::u8path`	Fix mandates	Remove now	pending...	pending...
D.24	C++11	C++20	P0883R2	atomic operations	No action	Partially Remove	pending...	pending...

One sign of the success we have had with previous papers is that roughly 70% of the subclauses in Annex D were added in the latest C++20 standard, and only one feature remains that was deprecated by the C++11 standard. Notably, that is the last remaining core feature that was deprecated prior to C++20.

D.1 Arithmetic conversion on enumerations [depr.arith.conv.enum]

First deprecated: C++20

Warning since: Clang 10

Warn on emum/float compare: MSVC 19.22

Warn comparing enums: Clang 3.3 (earliest I could test) gcc 4.1.2 (earliest I could test)

Not tested: EDG

This feature was deprecated as part of the effort to make the new spaceship operator *do the right thing*, adopted by paper P1120R0. It potentially impacts on code written against C++98 and later standards.

With the introduction of the 3-way comparison "spaceship" operator, there was a concern to avoid some implicit comparisons that might be lossy (due to rounding of floating point values) or giving up intended type safety (by using enums rather than integer types to indicate more than just a value). While the 3-way comparison operator is specified to reject such comparisons, the existing comparison operators were granted ongoing compatibility in these cases, but deprecated. It is likely that most, but not all, such usage relying on implicit conversion is a latent bug, and all such code would be clearer to folks reading the code if the implicit conversions were made explicit. Note that it is still possible to avoid explicit casts by using the unary + operator, forcing integral promotion.

While we would like to recommend the removal of these deprecated comparisons, bringing all the comparison operators into harmony for which types they interoperate on, the case for all uses being latent bugs is not as strong as for array comparisons (D.4), therefore it seems reasonable to allow another standard cycle encouraging compilers to issue deprecation warnings so that users can clean up their code. Hence, the strong recommendation is to do nothing for C++23, and strongly reconsider for C++26. The weak recommendation is to take decisive action and remove now, harmonizing the comparison operators.

Initial Review: telecon 2020/06/09

Concerns were raised about ongoing C compatibility. Any recommendation on removing deprecated support in D.1 will be forwarded to our WG14 liaison, to hopefully have a co-ordinated process for removing such features.

Concerns were raised over several issues of interoperating with different enumeration types, including expressions used to initialize global variables, and for array bounds. Similarly, metaprogramming idioms from C++98 would often use enumerations to denote result values (saving on storage for a static data member), and comparing such named values from different instantiations of the same template is the expected usage.

Another concern was raised for idioms for externally extending another enumeration without intruding on the original:

```
enum original { e_FIRST, ... , e_LAST };
enum extended { e_NEXT = e_LAST + 1, ... , e_MORE };
```

with the intent that the extended enumeration can interoperate with the original.

There seems fewer concerns for removing the interoperation with floating point types, although the was some discussion of the workarounds. There is concern that the "simple" forced promotion through unary operator `+` is too clever/cute, but that `static_cast`, or C-style casts, are verbose and risk converting to a different type than the previous implicit integer promotion.

There is an ongoing concern of gaining more specific feedback on implementation experience with the deprecation warning before making any change, and further concerns about gaining experience with any removal that might silently change behavior in SFINAE contexts.

Polls:

Q1: In C++23 Un-deprecate enum vs. enum?

SF	F	N	A	SA
1	2	7	6	4

No consensus

Q2: In C++23 Un-deprecate enum vs. floating-point?

SF	F	N	A	SA
8	1	4	8	6

No consensus

Q3: In C++23 remove the deprecated enum vs. enum facilities?

SF	F	N	A	SA
1	2	10	7	1

No consensus

Q4: In C++23 remove the deprecated enum vs. floating-point facilities?

SF	F	N	A	SA
7	10	4	0	1

Progress with this part

> **Strong recommendation:** Take no action.

> No change to draft.

> **Weak recommendation:** Remove this feature from C++23

7.4 Usual arithmetic conversions [expr.arith.conv]

1. Many binary operators that expect operands of arithmetic or enumeration type cause conversions and yield result types in a similar way. The purpose is to yield a common type, which is also the type of the result. This pattern is called the *usual arithmetic conversions*, which are defined as follows:
    1. — If either operand is of scoped enumeration type (9.7.1), no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.
    2. — If either operand is of unscoped enumeration type (9.7.1), and the other operand is of a different enumeration type or a floating-point type, the expression is ill-formed.
    3. — If either operand is of type `long double`, the other shall be converted to `long double`.
    4. — Otherwise, if either operand is `double`, the other shall be converted to `double`.
    5. — Otherwise, if either operand is `float`, the other shall be converted to `float`.
    6. — Otherwise, the integral promotions (7.3.6) shall be performed on both operands.[56] Then the following rules shall be applied to the promoted operands:
        1. — If both operands have the same type, no further conversion is needed.
        2. — Otherwise, if both operands have signed integer types or both have unsigned integer types, the operand with the type of lesser integer conversion rank shall be converted to the type of the operand with greater rank.
        3. — Otherwise, if the operand that has unsigned integer type has rank greater than or equal to the rank of the type of the other operand, the operand with signed integer type shall be converted to the type of the operand with unsigned integer type.
        4. — Otherwise, if the type of the operand with signed integer type can represent all of the values of the type of the operand with unsigned integer type, the operand with unsigned integer type shall be converted to the type of the operand with signed integer type.
        5. — Otherwise, both operands shall be converted to the unsigned integer type corresponding to the type of the operand with signed integer type.
2. ~~If one operand is of enumeration type and the other operand is of a different enumeration type or a floating-point type, this behavior is deprecated (D.1).~~

~~**D.1 Arithmetic conversion on enumerations [depr.arith.conv.enum]**~~

1. ~~The ability to apply the usual arithmetic conversions (7.4) on operands where one is of enumeration type and the other is of a different enumeration type or a floating-point type is deprecated. [Note: Three-way comparisons (7.6.8) between such operands are ill-formed. — *end note*] [*Example:*~~

    ```
    enum E1 { e };
    enum E2 { f };
    bool b = e <= 3.7;                  // deprecated
    int k = f - e;                      // deprecated
    auto cmp = e <=> f;                 // error
    ```

    ~~— *end example*]~~

No impact on library wording is anticipated by this removal.

Draft compatibility note for Annex C.

EWGI Review: Modified weak recommendation (remove just the enum/floating-point behavior)

7.4 Usual arithmetic conversions [expr.arith.conv]

1. Many binary operators that expect operands of arithmetic or enumeration type cause conversions and yield result types in a similar way. The purpose is to yield a common type, which is also the type of the result. This pattern is called the *usual arithmetic conversions*, which are defined as follows:
    1. — If either operand is of scoped enumeration type (9.7.1), no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.
    2. — Otherwise, if either operand is of unscoped enumeration type (9.7.1) and the other operand is of a floating-point type, the expression is ill-formed.
    3. — Otherwise, if either operand is of type `long double`, the other shall be converted to `long double`.
    4. — Otherwise, if either operand is `double`, the other shall be converted to `double`.
    5. — Otherwise, if either operand is `float`, the other shall be converted to `float`.
    6. — Otherwise, the integral promotions (7.3.6) shall be performed on both operands.[56] Then the following rules shall be applied to the promoted operands:
        1. — If both operands have the same type, no further conversion is needed.

2. — Otherwise, if both operands have signed integer types or both have unsigned integer types, the operand with the type of lesser integer conversion rank shall be converted to the type of the operand with greater rank.
3. — Otherwise, if the operand that has unsigned integer type has rank greater than or equal to the rank of the type of the other operand, the operand with signed integer type shall be converted to the type of the operand with unsigned integer type.
4. — Otherwise, if the type of the operand with signed integer type can represent all of the values of the type of the operand with unsigned integer type, the operand with unsigned integer type shall be converted to the type of the operand with signed integer type.
5. — Otherwise, both operands shall be converted to the unsigned integer type corresponding to the type of the operand with signed integer type.

2. If ~~one~~either operand is of unscoped enumeration type and the other operand is of a different enumeration type ~~or a floating-point type~~, this behavior is deprecated (D.1).

No impact on library wording is anticipated by this removal.

C.1.X Clause 7: Expressions [diff.cpp20.expr]

Affected subclause: 7.4 *Note for editors: [expr.arith.conv]*

Change: Unscoped enumerations do not implicitly promote to integral type in expressions with floating point types.

Rationale: ...

Effect on original feature: A valid C++ 2020 involving both an unscoped enumeration and a floating point value will be rejected as ill-formed in this International Standard. Either argument could be explicitly conerted with a cast, or the enumeration could be explicitly promoted to an integer with unary operator +, for no change of meaning since C++ 2020. [*Example:*

```
enum multipliers { gigaseconds = 1'000'000 };
double century_ish = 3.14 * gigaseconds;  // ill-formed; previously well-formed
double century_est = 3.14 * +gigaseconds;  // OK
```

—end example]

C.5.3 Clause 7: expressions [diff.expr]

Affected subclause: 7.4 *Note for editors: [expr.arith.conv]*

Change: Enumerations cannot be used in expressions with floating point types.

Rationale: ...

Effect on original feature: Deletion of semantically well-defined feature.

Difficulty of converting: Could be automated. Violations will be diagnosed by the C++ translator. The fix is to add a cast, or explicitly promote the `enum` object to an integer with unary operator `+`. For example:

How widely used: Common.

D.1 Arithmetic conversion on enumerations [depr.arith.conv.enum]

1. The ability to apply the usual arithmetic conversions (7.4) on operands where one is of unscoped enumeration type and the other is of a different enumeration type ~~or a floating-point type~~ is deprecated. [Note: Three-way comparisons (7.6.8) between such operands are ill-formed. *—end note*] [*Example:*

```
enum E1 { e };
enum E2 { f };
bool b = e <= 3.7;                  // deprecated
int k = f - e;                      // deprecated
auto cmp = e <=> f;                 // error
```

—end example]

D.2 Implicit capture of `*this` by reference [depr.capture.this]

First deprecated: C++20

Warning since: EDG 5.1, gcc 9, MSVC 19.22

No warnings: Clang

This feature was deprecated in C++20 by P0806R2. Its removal would potentially impact on programs written against C++11 or a later standard.

The concern addressed by the paper is that the implicit capture of `this` as a *reference* to data members on a default-capture that copies is surprising and often misleading. C++20 introduced the explicit capture of `this` as a pointer, or `*this` by value, to clearly disambiguate the different use cases. However, as the syntax to migrate to was introduced only in C++20, there was a clear desire for at least one standard retaining the implicit capture support as deprecated while users migrate their code at a time of their own choosing.

In the interest of clear code and fewer bugs from the surprising implicit defaults, the strong recommendation of this paper is to complete the work started in P0806 and remove the implicit capture from C++23. The weak recommendation is to retain deprecated support for another three years, and strongly reconsider removal in C++26. There is no suggestion to undeprecate.

Initial Review: telecon 2020/06/09

Concerns were raised about a lack of user experience with these, and a concern that the proposed fix for code that removes the deprecated syntax is actually ill-formed in earlier dialects of the language, prior to C++17. Removing this feature may make supporting long-lived code bases that support multiple C++ dialects awkward.

There was no enthusiasm to undeprecate the feature without a separate paper.

Polls:

Q1: In C++23, remove the deprecated implicit capture of *this by reference:

SF	F	N	A	SA
2	2	7	6	2

No consensus

Strong recommendation: Remove this feature from C++23

Draft core wording to enforce the removal (simply removing the deprecation notice is not enough).

~~**D.2 Implicit capture of `*this` by reference [depr.capture.this]**~~

1. ~~For compatibility with prior C++ International Standards, a *lambda-expression* with *capture-default* = (7.5.5.2) may implicitly capture `*this` by reference. [*Example*:~~

```
~~struct X {
  int x;
  void foo(int n) {
    auto f = [=]() { x = n; };        // deprecated: x means this->x, not a copy thereof
    auto g = [=, this]() { x = n; };  // recommended replacement
  }
};~~
```

~~— *end example*]~~

No impact on library wording is anticipated by this removal.

Draft compatibility note for Annex C.

Weak recommendation: Take no action yet, strongly reconsider removal in C++26.

No change to draft.

EWGI Review: Take no action for C++23.

D.3 Comma operator in subscript expressions [depr.comma.subscript]

First deprecated: C++20

Warning since: Clang 9, EDG 6.0, gcc 10, MSVC 19.25

> A more complete treatment of removing this feature and directly replacing it with support for multi-index array operators is presented in paper P2128R1 to which we might prefer to defer all handling of this topic.

This feature was deprecated for C++20 by paper P1161R3 to allow for a future extension in the language to support a simpler syntax for multiple-dimension arrays. Its removal would impact on code written against the C++98 and later standards, and potentially interoperating with C language headers.

There is a question over the value of removing this deprecated feature without also adding the extension for multiple-dimension arrays, which goes beyond the scope of this simpler review paper. The conservative view is that this would lead to code breakage for no clear benefit,so the strong recommendation is to do retain this feature as part of the overall review. The alternative view is that immediately applying a new meaning to existing syntax as part of an updated standard is a risk we could avoid by requiring at least one standard to make such code ill-formed, rather than merely deprecated. The weak recommendation is to remove this feature from C++23 to better enable repurposing the syntax in C++26, or beyond.

On as separate note regarding wording, we observe that the convention for deprecating a feature in the core language is to normatively define the deprecation in the core clauses (4-15) and then call back to that deprecation notice from Annex D. The C++20 standard has this backwards for this particular deprecation, with the two references to deprecation in the core clauses being *Note:*s, and the only normative text being in Annex D. At least one of the two *Note:*s should become normative, if the feature is retained.

Initial Review: telecon 2020/06/09

Several concerns were raised that might be better addressed for when EWG take a final decision. First, there is the existing paper, P2128, that seeks to both remove this deprecated feature, and replace it with a new meaning. One option is to delegate entirely to that paper, but loud concerns were also raised about changing meaning, and silently changing behavior, in the same standard. There is some support for removing the deprecated behavior now, and holding P2128 back until C++26. From the perspective of this paper, the notion is to proceed and treat P2128 independently.

Concerns were raised about implementation experience. While it is notable that all current compiler with C++20 support warn on this deprecation, some of those compilers are only a month or two old, and we get experience only when folks compile in the experimental C++20 modes. While we expect experience to grow, it is still minimal. It was also noted that the original paper performed a survey of popular open source libraries to assess impact, and hit only a vanishingly small set of matches in a deprecated Boost library: link

A final concern was raised about ongoing C compatibility, which will be forwarded to our WG14 liaison, to hopefully have a co-ordinated process for removing this feature.

Polls:

Q1: In C++23, remove the use of comma operator in subscript expressions?

SF	F	N	A	SA
8	9	2	1	1

Follow up with this direction

> **Strong recommendation:** Clean up wording.

7.6.1.1 Subscripting [expr.sub]

2. ~~[*Note:*~~ A comma expression (7.6.20) appearing as the *expr-or-braced-init-list* of a subscripting expression is deprecated; see D.3. ~~— *end note*]~~

7.6.20 Comma operator [expr.comma]

3. ~~[*Note:*~~ A comma expression (7.6.20) appearing as the *expr-or-braced-init-list* of a subscripting expression is deprecated; see D.3. ~~— *end note*]~~

> **Weak recommendation:** Remove this feature from C++23

7.6.1.1 Subscripting [expr.sub]

2. ~~[*Note:*~~ A comma expression (7.6.20) appearing as the *expr-or-braced-init-list* of a subscripting expression is ill-formed.~~deprecated; see D.3. — *end note*]~~

7.6.20 Comma operator [expr.comma]

3. [*Note:* A comma expression (7.6.20) appearing as the *expr-or-braced-init-list* of a subscripting expression is ill-formed~~deprecated; see D.3~~. *—end note*]

No impact on library wording is anticipated by this removal.

C.1.X Clause 7: Expressions [diff.cpp20.expr]

Affected subclause: 7.6.1.1 *Note for editors: [expr.sub]*

Change: Cannot parse unparenthesized comma expressions as subscript expressions.

Rationale: The old behavior was surprising for users familiar with other languages. It was removed to allow a future extension to support overloading the subscript operator for multiple arguments.

Effect on original feature: A valid C++ 2020 program using a comma expression as the argument to a subscript expression will be rejected as ill-formed in this International Standard. The intended comma expression can be enclosed in parentheses for no change of meaning since C++ 2020. [*Example:*

```
void f(int *a, int b, int c) {
    a[b,c];             // ill-formed; previously well-formed
    a[(b,c)];           // OK
}
```

Add an example of a DSEL

—end example]

C.5.3 Clause 7: expressions [diff.expr]

Affected subclause: 7.6.1.1 *Note for editors: [expr.sub]*

Change: Cannot parse unparenthesized comma expressions as subscript expressions.

Rationale: The old behavior was surprising for users familiar with other languages. It was removed to allow a future extension to support overloading the subscript operator for multiple arguments.

Effect on original feature: Deletion of semantically well-defined feature.

Difficulty of converting: Could be automated. Violations will be diagnosed by the C++ translator. The fix is to add parentheses. For example:

```
void f(int *a, int b, int c) {
    a[b,c];             // ill-formed; previously well-formed
    a[(b,c)];           // OK
}
```

How widely used: Rare.

~~**D.3 Comma operator in subscript expressions [depr.comma.subscript]**~~

1. ~~A comma expression (7.6.20) appearing as the *expr-or-braced-init-list* of a subscripting expression (7.6.1.1) is deprecated. [*Note:* A parenthesized comma expression is not deprecated. *—end note*] [*Example:*~~

```
void f(int *a, int b, int c) {
    a[b,c];                     // deprecated
    a[(b,c)];                   // OK
}
```

~~*—end example*]~~

EWGI Review: Accept the weak recommendation.

D.4 Array comparisons [depr.array.comp]

First deprecated: C++20

Warning since: Clang 10, MSVC 19.22

QoI Warning Clang 3.3 (earliest I could test)

No warnings: gcc

Not tested: EDG

This feature was deprecated as part of the effort to make the new spaceship operator *do the right thing*, adopted by paper P1120R0. It potentially impacts on code written against C++98 and later standards.

The deprecated comparison operator for arrays was not so much a deliberately designed feature, but accidental oversight that array-to-pointer decay would kick in, and so we compare whether two arrays are literally the same array at the same address, rather than whether two distinct arrays have the same contents. Identity tests are typically performed by explicitly taking the address of the objects we wish to validate; it would be highly unusual to rely on an implicit pointer decay to perform that task. The trick, if performed intentionally, offers no efficiency gain over explicitly taking the address of the array, but would fool a large number of subsequent code readers and reviewers who are not familiar with this trick. We do note that function comparison performs exactly the same decay, and users are not surprised that comparing functions is an identity test.

Given the likelihood that any usage of this comparison operator is a bug waiting to be detected, this could be a real concern for software reliability. Therefore, the strong recommendation is to remove this feature immediately from C++23. However, note for wording, that the comparison of an array with a pointer value is *not* deprecated in C++20, is reasonably idiomatic for practitioners of C++, and is intended to continue to be supported. This mimics the behavior of the spaceship operator.

Given the feature was so recently deprecated, the weak recommendation is to sit on the feature for another 3 years giving users more time to find (presumably benign) uses in their code and correct at a time of their choosing. We would expect to strongly recommend the removal again in another three years though, rather than consider undeprecation for such a feature.

Initial Review: telecon 2020/06/09

It was noted that comparison of pointer values is often unspecified, rather than well-defined, unless both arrays happen to be members of the same class, or are both elements of the same multi-dimensional array.

General agreement that this is a good opportunity to remove a landmine from the language that offers little benefit, even when correctly used as intended.

A concern was raised about ongoing C compatibility, which will be forwarded to our WG14 liaison, to hopefully have a co-ordinated process for removing this feature.

Polls:

Q1: In C++23, remove the deprecated use of array comparisons?

SF	F	N	A	SA
9	9	3	0	0

Follow up with this direction

Strong recommendation: Remove this feature from C++23

7.6.9 Relational operators [expr.rel]

1. The relational operators group left-to-right. [*Example:* `a<b<c` means `(a<b)<c` and not `(a<b)&&(b<c)`. — *end example*]

```
relational-expression : compare-expression
relational-expression < compare-expression
relational-expression > compare-expression
relational-expression <= compare-expression
relational-expression >= compare-expression
```

   The lvalue-to-rvalue (7.3.1)~~, array-to-pointer (7.3.2),~~ and function-to-pointer(7.3.3) standard conversions are performed on the operands. If at least one of the operands is a pointer, array-to-pointer conversions (7.3.2) are performed. ~~The comparison is deprecated if both operands were of array type prior to these conversions (D.4).~~
2. The converted operands shall have arithmetic, enumeration, or pointer type. The operators < (less than), > (greater than), <= (less than or equal to), and >= (greater than or equal to) all yield `false` or `true`. The type of the result is `bool`.
3. The usual arithmetic conversions (7.4) are performed on operands of arithmetic or enumeration type. If both operands are pointers, pointer conversions (7.3.11) and qualification conversions (7.3.5) are performed to bring them to their composite pointer type (7.2.2). After conversions, the operands shall have the same type.
4. The result of comparing unequal pointers to objects ...

7.6.10 Equality operators [expr.eq]

```
equality-expression :
    relational-expression
    equality-expression == relational-expression
    equality-expression != relational-expression
```

1. The == (equal to) and the != (not equal to) operators group left-to-right. The lvalue-to-rvalue (7.3.1)~~, array-to-pointer (7.3.2),~~ and function-to-pointer (7.3.3) standard conversions are performed on the operands. ~~The comparison is deprecated if both operands were of array type prior to these conversions (D.4).~~
2. If at least one of the operands is a pointer, array-to-pointer conversions (7.3.2), pointer conversions (7.3.11), function pointer conversions (7.3.13), and qualification conversions (7.3.5) are performed on both operands to bring them to their composite pointer type (7.2.2). ~~Comparing pointers is defined as follows:~~
3. The converted operands shall have arithmetic, enumeration, pointer, or pointer-to-member type, or type `std::nullptr_t`. The operators == and != both yield `true` or `false`, i.e., a result of type `bool`. In each case below, the operands shall have the same type after the specified conversions have been applied.
4. Comparing pointers is defined as follows:
    1. — If one pointer represents the address of a complete object, and another pointer represents the address one past the last element of a different complete object,[79] the result of the comparison is unspecified.
    2. — Otherwise, if the pointers are both null, both point to the same function, or both represent the same address (6.8.2), they compare equal.
    3. — Otherwise, the pointers compare unequal.
5. If at least one of the operands is a pointer to member, ...

No impact on library wording is anticipated by this removal.

C.1.X Clause 7: Expressions [diff.cpp20.expr]

Affected subclause: 7.6.9 and 7.6.10 *Note for editors: [expr.rel] and [expr.eq]*

Change: Cannot compare two objects of array type.

Rationale: The old behavior was confusing, as it did not compare the contents of the two arrays, but compare their addresses. Depending on context, this would either report whether the two arrays were the same object, or have an unspecified result.

Effect on original feature: A valid C++ 2020 program directly comparing two array objects will be rejected as ill-formed in this International Standard. [*Example:*

```
int arr1[5];
int arr2[5];
bool same = arr1 ==  arr2;       // ill-formed; previously well-formed
bool idem = arr1 == +arr2;       // compare addresses, unspecified result
```

—end example]

C.5.3 Clause 7: expressions [diff.expr]

Affected subclause: 7.6.9 and 7.6.10 *Note for editors: [expr.rel] and [expr.eq]*

Change: Cannot compare two objects of array type.

Rationale: The old behavior was confusing, as it did not compare the contents of the two arrays, but compare their addresses. Depending on context, this would either report whether the two arrays were the same object, or have an unspecified result.

Effect on original feature: Deletion of feature with unspecified behavior.

Difficulty of converting: Violations will be diagnosed by the C++ translator.

How widely used: Rare. In the cases where the result is well defined, it is reporting whether both arguments are the same object, using the same name.

~~**D.4 Array comparisons [depr.array.comp]**~~

1. ~~Equality and relational comparisons (7.6.10, 7.6.9) between two operands of array type are deprecated. [*Note:* Three-way comparisons (7.6.8) between such operands are ill-formed. — *end note*] [*Example:*~~

```
int arr1[5];
int arr2[5];
bool same = arr1 == arr2;        // deprecated, same as &arr1[0] == &arr2[0],
```

~~// does not compare array contents~~
~~auto cmp = arr1 <=> arr2; // error~~

~~*—end example*~~]

Weak recommendation: Take no action.

No change to draft.

EWGI Review: Accept the strong recommendation.

D.5 Deprecated volatile types [depr.volatile.type]

First deprecated: C++20

Warning since: Clang 10, EDG 6.0, gcc 10

No warnings: MSVC

The `volatile` keyword is an original part of the C legacy for C++, and describes constraints on programs intended to model hardware changing values beyond the program's control. As this entered the type system of C++, certain interactions were discovered to be troublesome, and latent bugs that could be detected at the time of program translation go unreported. The paper breaks down each context where the `volatile` keyword can be used, and deprecated those uses that are unconditionally dangerous, or serve no good purpose. This paper is the first opportunity to go further, and remove those use cases after 3 years of giving users deprecation warnings.

A quick micro-analysis suggests the main concerns of the first two paragraphs are read/modify/write operations, where by the nature of volatile objects, the value being rewritten may have changed since read and modified. This kind of pattern is most likely in old (pre-C++11) code using `volatile` as a poor proxy for atomic. Since we will have over a decade of real atomic support in the language when C++23 ships, it could be desirable to further encourage such code (when compiled in the latest dialect) to adapt to the memory model and its stronger guarantees.

The third paragraph addresses function arguments and return values. These are temporary or elided objects created entirely by the compiler, and guaranteed to not display the uncertainty of value implied by the `volatile` keyword. As such, any use is redundant and misleading, so it would be helpful to remove this facility sooner rather than later, and have one fewer oddity to teach when learning (and understanding) the language. The biggest concern would be for compatibility with C code, that may still use this feature in its headers. To mitigate, me may consider removing `volatile` function parameters and return values for only functions with `extern "C++"` linkage.

The fourth paragraph considers volatile qualifier in structured bindings, and can affect only code written since C++17, that will have been deprecated as long as it was non-deprecated when C++23 is published. It would be good to remove this now, before more deprecated code is written.

As outright removal is introducing potential breakage into programs that have been compiling successfully since C++98, this paper strongly recommends to hold this feature in Annex D as deprecated for another 3 years, and strongly consider taking further action for C++26. However, as there is the potential to diagnose real bugs for users to fix, the weak recommendation is to remove these deprecated use cases immediately from C++23. It would also be possible to review each of the 4 noted usages separately, and remove only the features with lowest risk from removal, notably paragraphs 3 and 4.

Strong recommendation: Take no action.

No change to draft.

Weak recommendation: Remove this feature from C++23

Note that the core terms *arithmetic type* and *pointer type* refer only to cv-unqualified types, and cv-qualifiers are incorporated only for the larger classification of *scalar types*; therefore, much of the deprecated wording in clause 7 is already redundant - this will be core issue 2448. In principle, we need to add words to enable the feature that C++20 intended to deprecate! The wording issue goes back to C++98 though, as current implementations provide the intended deprecated behavior, not a literal interpretation of the standard.

7.6.1.5 Increment and decrement [expr.post.incr]

1. The value of a postfix ++ expression is the value of its operand. [*Note:* The value obtained is a copy of the original value. *—end note*] The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type other than ~~cv~~ `bool`, or a pointer to a complete object type. ~~An operand with volatile-qualified type is deprecated; see D.5.~~ The value of ...

7.6.2.2 Increment and decrement [expr.pre.incr]

1. The operand of prefix `++` is modified (3.1) by adding `1`. The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type other than ~~*cv*~~ `bool`, or a pointer to a completely-defined object type. ~~An operand with volatile-qualified type is deprecated; see D.5.~~ The result is ...

7.6.19 Assignment and compound assignment operators [expr.ass]

5. ~~A simple assignment whose left operand is of a `volatile`-qualified type is deprecated (D.5) unless the (possibly parenthesized) assignment is a discarded-value expression or an unevaluated operand.~~

> Draft a replacement paragraph so that assignment to `volatile`-qualified types remains valid, but only through discarded-value expressions. If not an ABI consideration, could we make the built-in assignment operators for `volatile`-qualified types return `void`?

6. The behavior of an expression of the form `E1` *op*`=` `E2` is equivalent to `E1 = E1` *op* `E2` except that `E1` is evaluated only once. ~~Such expressions are deprecated if `E1` has `volatile`-qualified type; see D.5.~~ For `+=` and `-=`, `E1` shall either have arithmetic type or be a pointer to a possibly cv-qualified completely-defined object type. In all other cases, `E1` shall have arithmetic type.

9.3.3.5 Functions [dcl.fct]

> It is expected that removing support for volatile-qualified function parameters will have ripples more widely through the standard, and that potentially cv-qualified types may become potentially const-qualified types in a number of places, after a deeper audit.

4. The *parameter-declaration-clause* determines the arguments that can be specified, and their processing, when the function is called. [*Note:* The *parameter-declaration-clause* is used to convert the arguments specified on the function call; see 7.6.1.2. *—end note*] If the *parameter-declaration-clause* is empty, the function takes no arguments. A parameter list consisting of a single unnamed parameter of non-dependent type `void` is equivalent to an empty parameter list. Except for this special case, a parameter shall not have type *cv* `void`. A parameter ~~with~~shall not have a volatile-qualified type ~~is deprecated; see D.5~~. If the *parameter-declaration-clause* terminates with an ellipsis or a function parameter pack (13.7.3), the number of arguments shall be equal to or greater than the number of parameters that do not have a default argument and are not function parameter packs. Where syntactically correct and where "..." is not part of an *abstract-declarator*, ", ..." is synonymous with "...". [*Example:* The declaration

```
int printf(const char*, ...);
```

declares a function that can be called with varying numbers and types of arguments.

```
printf("hello world");
printf("a=%d b=%d", a, b);
```

However, the first argument must be of a type that can be converted to a `const char*`. *—end example*] [*Note:* The standard header `<cstdarg>` (17.13.1) contains a mechanism for accessing arguments passed using the ellipsis (see 7.6.1.2 and 17.13). *—end note*]

12. ~~A volatile-qualified return type is deprecated; see D.5.~~ A return type shall not be volatile-qualified. [*Note:* References to volatile-qualified types are permitted, as reference types do not have a top level cv-qualifier. *—end note*]

9.6 Structured binding declarations [dcl.struct.bind]

> As a reminder, the grammar for a structured binding is the following *simple-declaration*:
>
> *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq ref-qualifier*$_{opt}$ `[` *identifier-list* `]` *initializer* `;`

1. A structured binding declaration introduces the identifiers v_0, v_1, v_2, . . . of the *identifier-list* as names (6.4.1) of structured bindings. Let *cv* denote the *cv-qualifiers* in the *decl-specifier-seq* and *S* consist of the *storage-class-specifiers* of the *decl-specifier-seq* (if any). A *cv* that includes `volatile` is deprecated; see D.5. First, a variable with a unique name `e` is introduced. If the *assignment-expression* in the *initializer* has array type `A` and no *ref-qualifier* is present, `e` is defined by

> *attribute-specifier-seq*$_{opt}$ *S cv* `A e ;`

and each element is copy-initialized or direct-initialized from the corresponding element of the *assignment-expression* as specified by the form of the *initializer*. Otherwise, `e` is defined as-if by

> *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq ref-qualifier*$_{opt}$ `e` *initializer* `;`

where the declaration is never interpreted as a function declaration and the parts of the declaration other than the *declarator-id* are taken from the corresponding structured binding declaration. The type of the *id-expression* `e` is called

`E`. [*Note:* `E` is never a reference type (7.2). *—end note*]

> Ideally, this whole paragraph should be rewritten to avoid introducing a *cv* that can never include the `volatile` qualifier.

2. If the *initializer* refers to one of the names introduced by the structured binding declaration, the program is ill-formed.
3. If `E` is an array type with element type `T`, the number of elements in the *identifier-list* shall be equal to the number of elements of `E`. Each v_i is the name of an lvalue that refers to the element *i* of the array and whose type is `T`; the referenced type is `T`. [*Note:* The top-level cv-qualifiers of `T` are *cv*. *—end note*] [*Example:*

```
auto f() -> int(&)[2];
auto [ x, y ] = f();    // x and y refer to elements in a copy of the array return value
auto& [ xr, yr ] = f(); // xr and yr refer to elements in the array referred to by f's return value
```

—end example]
4. Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete class type with a member named `value`, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression. Let `i` be an index prvalue of type `std::size_t` corresponding to v_i. The *unqualified-id* `get` is looked up in the scope of `E` by class member access lookup (6.5.5), and if that finds at least one declaration that is a function template whose first template parameter is a non-type parameter, the *initializer* is `e.get<i>()`. Otherwise, the *initializer* is `get<i>(e)`, where `get` is looked up in the associated namespaces (6.5.2). In either case, `get<i>` is interpreted as a *template-id*. [*Note:* Ordinary unqualified lookup (6.5.1) is not performed. *—end note*] In either case, `e` is an lvalue if the type of the entity `e` is an lvalue reference and an xvalue otherwise. Given the type T_i designated by `std::tuple_element<i, E>::type` and the type U_i designated by either T_i`&` or T_i`&&`, where U_i is an lvalue reference if the initializer is an lvalue and an rvalue reference otherwise, variables are introduced with unique names r_i as follows:

S U_i r_i `=` *initializer* `;`

Each v_i is the name of an lvalue of type T_i that refers to the object bound to r_i; the referenced type is T_i.
5. Otherwise, all of `E`'s non-static data members shall be direct members of E or of the same base class of `E`, well-formed when named as `e.name` in the context of the structured binding, `E` shall not have an anonymous union member, and the number of elements in the *identifier-list* shall be equal to the number of non-static data members of `E`. Designating the non-static data members of E as m_0, m_1, m_2, ... (in declaration order), each v_i is the name of an lvalue that refers to the member m_i of `e` and whose type is *cv* T_i, where T_i is the declared type of that member; the referenced type is *cv* T_i. The lvalue is a bit-field if that member is a bit-field. [*Example:*

```
struct S { int x1 : 2; volatile double y1; };
S f();
const auto [ x, y ] = f();
```

The type of the *id-expression* `x` is "`const int`", the type of the *id-expression* `y` is "`const volatile double`". *—end example*]

> Note that the use of `volatile` in this example remains valid for structured bindings, even after the removal of the deprecated parts.

12.7 Built-in operators [over.built]

3. ~~In the remainder of this subclause, *vq* represents either `volatile` or no cv-qualifier.~~
4. For every ~~pair (`T`, *vq*), where `T` is an~~ arithmetic type other than `bool`, there exist candidate operator functions of the form

```
vq T & operator++(vq T &);
 T operator++(vq T &, int);
```

5. For every ~~pair (`T`, *vq*), where `T` is an~~ arithmetic type other than `bool`, there exist candidate operator functions of the form

```
vq T & operator--(vq T &);
 T operator--(vq T &, int);
```

6. For every ~~pair (`T`, *vq*)~~type `T`, where `T` is a cv-qualified or cv-unqualified object type, there exist candidate operator functions of the form

```
T*vq& operator++(T*vq&);
T*vq& operator--(T*vq&);
T* operator++(T*vq&, int);
T* operator--(T*vq&, int);
```

21. For every ~~triple (`L`, *vq*, `R`)~~pair (`L`, `R`), where `L` is an arithmetic type, and `R` is a floating-point or promoted integral type, there exist candidate operator functions of the form

```
vq L & operator=(vq L &, R);
void operator=(volatile L &, R);
```

```
vq L & operator*=(vq L &, R);
vq L & operator/=(vq L &, R);
vq L & operator+=(vq L &, R);
vq L & operator-=(vq L &, R);
```

22. For every pair (T, vq)type T, there exist candidate operator functions of the form

```
T*vq& operator=(T*vq&, T*);
void operator=(T* volatile &, T*);
```

23. For every pair (T, vq), where T is an enumeration or pointer-to-member type, there exist candidate operator functions of the form

```
vq T & operator=(vq T &, T);
void operator=(volatile T &, T);
```

24. For every pair (T, vq)type T, where `T` is a cv-qualified or cv-unqualified object type, there exist candidate operator functions of the form

```
T*vq& operator+=(T*vq&, std::ptrdiff_t);
T*vq& operator-=(T*vq&, std::ptrdiff_t);
```

25. For every triple (L, vq, R)pair (L, R), where `L` is an integral type, and `R` is a promoted integral type, there exist candidate operator functions of the form

```
vq L & operator%=(vq L &, R);
vq L & operator<<=(vq L &, R);
vq L & operator>>=(vq L &, R);
vq L & operator&=(vq L &, R);
vq L & operator^=(vq L &, R);
vq L & operator|=(vq L &, R);
```

D.5 Deprecated volatile types [depr.volatile.type]

1. Postfix ++ and -- expressions (7.6.1.5) and prefix ++ and -- expressions (7.6.2.2) of volatile-qualified arithmetic and pointer types are deprecated.

```
[Example:
  volatile int velociraptor;
  ++velociraptor;                       // deprecated
 — end example]
```

2. Certain assignments where the left operand is a volatile-qualified non-class type are deprecated; see 7.6.19.

```
[Example:
  int neck, tail;
  volatile int brachiosaur;
  brachiosaur = neck;                   // OK
  tail = brachiosaur;                   // OK
  tail = brachiosaur = neck;            // deprecated
  brachiosaur += neck;                  // deprecated
  brachiosaur = brachiosaur + neck;     // OK
 — end example]
```

3. A function type (9.3.3.5) with a parameter with volatile-qualified type or with a volatile-qualified return type is deprecated.

```
[Example:
  volatile struct amber jurassic();                                 // deprecated
  void trex(volatile short left_arm, volatile short right_arm);     // deprecated
  void fly(volatile struct pterosaur* pteranodon);                  // OK
 — end example]
```

4. A structured binding (9.6) of a volatile-qualified type is deprecated.

```
[Example:
  struct linhenykus { short forelimb; };
  void park(linhenykus alvarezsauroid) {
    volatile auto [what_is_this] = alvarezsauroid;                  // deprecated
    // ...
  }
 — end example]
```

Draft compatibility note for Annex C.

EWGI Review: To be determined...

D.6 Redeclaration of static constexpr data members [depr.static.constexpr]

First deprecated: C++17

Warning since: *(none)*

No warnings: Clang EDG gcc MSVC

Static constexpr data members were added to the language in C++11, as part of the initial constexpr feature. However, they still required a definition of the member outside the class. When inline variables were added in C++17 (P0386R2) then static constexpr data members became implicitly inline, and the external definition became redundant, so was deprecated.

When we considered this feature for removal in C++20, it was thought the change was too recent for users to have time to adjust, and recommended we reconsider removal in C++23 (or later). By the time we publish C++23, this feature will have been deprecated for exactly as long as it was non-deprecated, 6 years. The suggested fix to make code confirming again is simple: remove the offending definition as it is redundant; no other changes are necessary. For code that wishes to be compatible across a range of standard dialects, the definition can be guarded by the preprocessor:

```
#if __cplusplus < 201703
constexpr Type Class::Member;
#endif
```

If we were to remove this feature early in the C++23 cycle, and compiler vendors expose this through their experimental C++23 support, we would get sufficient feedback if removal of this feature were a step too far, too soon. The strong recommendation is to remove this feature from C++23. Note, however, that at the time of publishing this paper, no current compiler warns of this deprecation, in either C++17 or C++20 build modes.

The feature seems relatively small and harmless; it is not clear that there is a huge advantage to removing it immediately from C++23, although it would be one less corner case to teach. The weak recommendation is to retain this feature, advocating removal in C++26, by which time the community will have had most of a decade to clean up old C++11/14 code.

Strong recommendation: remove this facility from C++23.

6.1 Declarations and definitions [basic.def]

2. A declaration is a *definition* unless
    1. — it declares a function without specifying the function's body (11.4),
    2. — it contains the `extern` specifier (9.2.1) or a *linkage-specification*[22] (9.11) and neither an *initializer* nor a *function-body*,
    3. — it declares a non-inline static data member in a class definition (11.4, 11.4.8),
    4. ~~— it declares a static data member outside a class definition and the variable was defined within the class with the `constexpr` specifier (this usage is deprecated; see D.6),~~
    5. — it is introduced by an *elaborated-type-specifier* (11.3),
    6. — it is an *opaque-enum-declaration* (9.7.1),
    7. — it is a *template-parameter* (13.2),
    8. — it is a *parameter-declaration* (9.3.3.5) in a function declarator that is not the *declarator* of a *function-definition*,
    9. — it is a `typedef` declaration (9.2.3),
    10. — it is an *alias-declaration* (9.2.3),
    11. — it is a *using-declaration* (9.9),
    12. — it is a *deduction-guide* (13.7.1.2),
    13. — it is a *static_assert-declaration* ((9.1),
    14. — it is an *attribute-declaration* ((9.1),
    15. — it is an *empty-declaration* (9.1),
    16. — it is a *using-directive* (9.8.3),
    17. — it is a *using-enum-declaration* (9.7.2),
    18. — it is a *template-declaration* (13.1) whose *template-head* is not followed by either a *concept-definition* or a declaration that defines a function, a class, a variable, or a static data member.
    19. — it is an explicit instantiation declaration (13.9.2), or
    20. — it is an explicit specialization (13.9.3) whose *declaration* is not a definition.

11.4.8.2 Static data members [class.static.data]

4. If a non-volatile non-inline `const` static data member is of integral or enumeration type, its declaration in the class definition can specify a *brace-or-equal-initializer* in which every *initializer-clause* that is an *assignment-expression* is a constant expression (7.7). The member shall still be defined in a namespace scope if it is odr-used (6.3) in the program and the namespace scope definition shall not contain an *initializer*. An inline static data member may be defined in the class definition and may specify a *brace-or-equal-initializer*. ~~If the member is declared with the `constexpr` specifier, it may be redeclared in namespace scope with no initializer (this usage is deprecated; see D.6).~~ Declarations of other static data members shall not specify a *brace-or-equal-initializer*.

~~**D.6 Redeclaration of static constexpr data members [depr.static.constexpr]**~~

1. ~~For compatibility with prior C++ International Standards, a `constexpr` static data member may be redundantly redeclared outside the class with no initializer. This usage is deprecated. [*Example*:~~

```
struct A {
  static constexpr int n = 5; // definition (declaration in C++ 2014)
};

constexpr int A::n;           // redundant declaration (definition in C++ 2014)
```

~~— *end example*]~~

No impact on library wording is anticipated by this removal.

Draft compatibility note for Annex C.

Weak recommendation: Take no action yet, consider again for C++26.

No change to draft.

EWGI Review: To be determined...

D.7 Non-local use of TU-local entities [depr.local]

First deprecated: C++20

Warning since: *(none)*

No warnings: Clang EDG gcc MSVC

This feature was deprecated as part of the effort to cleanly introduce modules into the language, and was adopted by paper P1815R2. It potentially impacts on code written against C++98 and later standards.

This feature was deprecated only at the final meeting of the C++20 development cycle, and at the time this paper is written, there is no experience with how much code has been impacted by this deprecation. As such, it is too early to make any recommendation for a change with respect to this feature.

Strong recommendation: take no action.

No change to draft.

Weak recommendation: take no action.

No change to draft.

EWGI Review: To be determined...

D.8 Implicit declaration of copy functions [depr.impldec]

First deprecated: C++11

Warning since: Clang 10, gcc 9

No warnings: MSVC

Not tested: EDG

Relevant issues: Core #2132

Core issue #2132 was discussed on the EWG telecon of 2020/04/29. The consensus on the call was that this is a contentious issue that should be handled by a paper in its own right.

This feature was deprecated towards the end of the C++11 development cycle by paper N3203 and was heavily relied on prior to that. That said, the deprecated features are implicitly deleted if any move operations are declared, and users have become familiar with this idiom over the last decade, and may now find the older deprecated behavior jarring.

When we reviewed this facility for C++20, these was something of an impasse. There was concern that removing this feature *ever* would break too much code to ever consider it. Conversely, there was no way to achieve a consensus to undeprecate these features. Finally, it was noted that no compiler had ever issued a deprecation warning for these constructs, and all compiler implementers present opined that they would never implement such a warning, as it would flag far too much code to be a usable warning.

Since then, gcc has indeed implemented the warning, and incorporated into its `-Wall` setting, the flag to turn on all the warnings that have a vanishingly small false-positive rate. Similarly, Clang 10 has released with the same warning enabled since R0 of this document was published. The feedback from that exercise was that coupling the destructor into the rule to delete copy operations was indeed far too noisy, and unlikely to be realizable in production code. Legitimate use case to implement a destructor unrelated to copy operation occur too frequently in practice, such as a simple act of logging or asserting invariants at the end of an object's lifetime. However, the coupling of the two copy operations produced a much lower (and tractable) number of warnings, and found real bugs along with a number of false positives.

The workaround for the issue in the vast majority of cases is to simply declare the other copy member in the class definition as `= default` so the fix is (mostly) simple, easily hinted, and modern tools can even offer to apply the fix for you as part of the warning. There may be a concern that the fix relies on a C++11 feature, and so requires more work for code maintaining C++03 compatibility. This is true, but for C++03 there is no functional difference between generating the default and writing it by hand. For C++11 the potential difference is that a user-provided copy operation is no longer trivial, but the distinction of decomposing PODs into trivial operations and standard layout types matters only for C++11 onwards, as user-providing either operation causes the type to not be a POD in C++03.

One unusual case for consideration is an idiom for defining iterator types where the `const_iterator` can be implicitly constructed from the plain `iterator`. One way to achieve this is for the two iterators to be instantiations of the same class template, but with a `const` qualified type argument for the `const_iterator`, and use a metafunction to produce an alias for the `const_iterator` in both cases. An implicit constructor declared with this aliased name will be a copy constructor for the plain `iterator`, or an additional overloaded constructor for the `const_iterator`. However, we cannot declare both constructors, as in the case of the plain `iterator` that would be a duplicate declaration, so for the case of `const_iterator` we *must* rely on the implicit copy constructor declaration. As the declared constructor is not the copy constructor for all instantiations, it cannot be defaulted. In a common implementation of this idiom we similarly rely on the implicitly declared copy-assignment operator, which would now have to be user-declared. However, due to the declaration matching the signature of the copy assignment operator for only the `iterator` instantiations, we cannot simply default this new declaration, and must explicitly provide a definition too.

```
template<class T>
class MyIterator {
   T * d_ptr = nullptr;

 public:
   using base_iterator = MyIterator<std::remove_const_t<T>>;

   MyIterator() = default;
   explicit MyIterator(T * ptr) : d_ptr{ptr} {}

   MyIterator(base_iterator const & it) : d_ptr{it.operator->()} {}
     // This is the copy constructor if 'T' is not 'const'-qualified,
     // otherwise the copy constructor is implicitly declared.

   MyIterator& operator=(base_iterator const & it) {
     // This is the copy-assignment operator if 'T' is not 'const'-qualified,
     // otherwise the copy-assignment operator is implicitly declared.
     d_ptr = it.operator->();
     return *this;
   }

   T * operator->() const noexcept { return d_ptr; }

   // ...
};
```

The strong recommendation of this paper is mixed: it is time to enforce the rule coupling the copy operations in the same manner as the move operations are coupled, and remove the legacy auto-generation if just one is user-defined. However, at the same time, we should uncouple the destructor from this rule, undeprecating its impact on copy operations. We might consider uncoupling the destructor from move operations for consistency, but that would be a separate paper.

The weak recommendation is to continue another standard cycle with this feature deprecated as-is, but consider what actions and research might demonstrate we are ready to remove this coupling for C++26.

Strong recommendation: take decisive action early in the C++23 development cycle, so early adopters can shake out the remaining cost of updating old code. Note that this would be both an API and ABI breaking change, and it would be good to set that precedent early if we wish to allow such breakage in C++23.

11.4.4.2 Copy/move constructors [class.copy.ctor]

6. ~~If the class definition does not explicitly declare a copy constructor, a non-explicit one is declared *implicitly*. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy constructor is defined as deleted; otherwise, it is defined as defaulted (9.5). The latter case is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor.~~

   If the definition of a class `X` does not explicitly declare a copy constructor, a non-explicit one will be implicitly declared as defaulted if and only if

   1. — `X` does not have a user-declared copy assignment operator,
   2. — `X` does not have a user-declared move constructor, and
   3. — `X` does not have a user-declared move assignment operator.

   > User declared destructors are deliberately omitted from this list. This wording undeprecates that concern - we could insert additional wording to retain the deprecated status when the class has a user-declared destructor. Note that we probably want the term "user-provided" as a user-declared `= default` destructor should not be a concern.

11.4.5 Copy/move assignment operator [class.copy.assign]

2. ~~If the class definition does not explicitly declare a copy assignment operator, one is declared *implicitly*. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy assignment operator is defined as deleted; otherwise, it is defined as defaulted (9.5). The latter case is deprecated if the class has a user-declared copy constructor or a user-declared destructor.~~

   If the definition of a class `X` does not explicitly declare a copy assignment operator, one will be implicitly declared as defaulted if and only if

   1. — `X` does not have a user-declared copy constructor,
   2. — `X` does not have a user-declared move constructor, and
   3. — `X` does not have a user-declared move assignment operator.

   > User declared destructors are deliberately omitted from this list. This wording undeprecates that concern - we could insert additional wording to retain the deprecated status when the class has a user-declared destructor. Note that we probably want the term "user-provided" as a user-declared `= default` destructor should not be a concern.

3. The implicitly-declared copy assignment operator for a class `X` will have the form...

~~D.8 Implicit declaration of copy functions [depr.impldec]~~

1. ~~The implicit definition of a copy constructor as defaulted is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor. The implicit definition of a copy assignment operator as defaulted is deprecated if the class has a user-declared copy constructor or a user-declared destructor (15.4, 15.8). In a future revision of this International Standard, these implicit definitions could become deleted (11.4).~~

Draft compatibility note for Annex C.

Weak recommendation: take no action now, and plan to remove this feature in the next revision of the standard that will permit both ABI and API breakage.

No change to draft.

EWGI Review: To be determined...

D.9 C headers [depr.c.headers]

First deprecated: C++98

The basic C library headers are an essential compatibility feature, and not going anywhere anytime soon. However, there are certain C++ specific counterparts that do not bring value, particularly where the corresponding C header's job is to supply macros that masquerade as keywords already present in the C++ language.

One possibility to be more aggressive here, following the decision to not adopt *all* C11 headers as part of the C++ mapping to C, would be to remove those same C headers from the subset that must be shipped with a C++ compiler. The corresponding change for the C++ `<cstd*>` was accepted for C++20. This would not prevent those headers being supplied, as part of a full C implementation, but would indicate that they have no value to a C++ system.

Finally, it seems clear that the C headers will be retained essentially forever, as a vital compatibility layer with C and POSIX. After two decades of deprecation, not a single compiler provides a deprecation warning on including these headers, and it is difficult to imagine circumstances where they would, as any use of a third party C library by a C++ project is almost guaranteed to include one of these headers, at least indirectly. Therefore, it seems appropriate to undeprecate the headers, and find a home for them as a compatibility layer in the main standard. It is also possible that we will want to explore a different approach in the future once modules are part of C++, and the library is updated to properly take advantage of the new language feature.

The strong recommendation of this paper is to undeprecate the C library as an essential compatibility layer with the rest of the world, optionally removing the vacuous headers.

The weak recommendation is to remove these entirely from C++23: these headers are already owned and specified by at least two other ISO committees, and while C compatibility is an essential feature for any C++ tool chain sitting on POSIX, it may be over-specification demanding what should be platform implementation details for other environments - as noted when C++17 chose to not require the C `<atomic.h>`.

Strong recommendation(A): Remove the vacuous headers, then undeprecate the remaining **[depr.c.headers]** and move directly into 16.5.5.2 **[res.on.headers]**.

16.5.5.2.1 C standard library headers [c.headers]

1. For compatibility with the C standard library, the C++ standard library provides the C headers shown in Table 147.

Table 147 — C headers [tab:depr.c.headers]

`<assert.h>`	`<inttypes.h>`	`<signal.h>`	`<stdio.h>`	`<wchar.h>`
~~`<complex.h>`~~	~~`<iso646.h>`~~	~~`<stdalign.h>`~~	`<stdlib.h>`	`<wctype.h>`
`<ctype.h>`	`<limits.h>`	`<stdarg.h>`	`<string.h>`	
`<errno.h>`	`<locale.h>`	~~`<stdbool.h>`~~	~~`<tgmath.h>`~~	
`<fenv.h>`	`<math.h>`	`<stddef.h>`	`<time.h>`	
`<float.h>`	`<setjmp.h>`	`<stdint.h>`	`<uchar.h>`	

2. Every C header, each of which has a name of the form `<name.h>`, behaves as if each name placed in the standard library namespace by the corresponding `<cname>` header is placed within the global namespace scope, except for the functions described in 26.8.6, the declarations of `std::nullptr_t` (17.2.3) and `std::byte` (17.2.5), and the functions and function templates described in 17.2.5. It is unspecified whether these names are first declared or defined within namespace scope (6.4.6) of the namespace `std` and are then injected into the global namespace scope by explicit using-declarations (9.9).
3. [*Example:* The header `<cstdlib>` assuredly provides its declarations and definitions within the namespace `std`. It may also provide these names within the global namespace. The header `<stdlib.h>` assuredly provides the same declarations and definitions within the global namespace, much as in the C Standard. It may also provide these names within the namespace `std`. —*end example*]

~~**D.9 C headers [depr.c.headers]**~~

1. ~~For compatibility with the C standard library, the C++ standard library provides the *C headers* shown in Table 147.~~

~~Table 147 — C headers [tab:depr.c.headers]~~

~~`<assert.h>`~~	~~`<inttypes.h>`~~	~~`<signal.h>`~~	~~`<stdio.h>`~~	~~`<wchar.h>`~~
~~`<complex.h>`~~	~~`<iso646.h>`~~	~~`<stdalign.h>`~~	~~`<stdlib.h>`~~	~~`<wctype.h>`~~
~~`<ctype.h>`~~	~~`<limits.h>`~~	~~`<stdarg.h>`~~	~~`<string.h>`~~	
~~`<errno.h>`~~	~~`<locale.h>`~~	~~`<stdbool.h>`~~	~~`<tgmath.h>`~~	
~~`<fenv.h>`~~	~~`<math.h>`~~	~~`<stddef.h>`~~	~~`<time.h>`~~	
~~`<float.h>`~~	~~`<setjmp.h>`~~	~~`<stdint.h>`~~	~~`<uchar.h>`~~	

~~**D.9.1 Header `<complex.h>` synopsis [depr.complex.h.syn]**~~

~~`#include <complex>`~~

1. ~~The header `<complex.h>` behaves as if it simply includes the header `<complex>` (26.4.1).~~
2. ~~[*Note:* Names introduced by `<complex>` in namespace `std` are not placed into the global namespace scope by `<complex.h>`. —*end note*]~~

~~**D.9.2 Header `<iso646.h>` synopsis [depr.iso646.h.syn]**~~

1. ~~The C++ header `<iso646.h>` is empty. [*Note:* `and`, `and_eq`, `bitand`, `bitor`, `compl`, `not_eq`, `not`, `or`, `or_eq`, `xor`, and `xor_eq` are keywords in this International Standard (5.11). —*end note*]~~

~~**D.9.3 Header `<stdalign.h>` synopsis [depr.stdalign.h.syn]**~~

~~`#define __alignas_is_defined 1`~~

1. ~~The contents of the C++ header `<stdalign.h>` are the same as the C standard library header `<stdalign.h>`, with the following changes: The header `<stdalign.h>` does not define a macro named alignas.~~

   ~~See also: ISO C 7.15~~

~~D.9.4 Header `<stdbool.h>` synopsis [depr.stdbool.h.syn]~~

~~`#define __bool_true_false_are_defined 1`~~

1. ~~The contents of the C++ header `<stdbool.h>` are the same as the C standard library header `<stdbool.h>`, with the following changes: The header `<stdbool.h>` does not define macros named `bool`, `true`, or `false`.~~

   ~~See also: ISO C 7.18~~

~~D.9.5 Header `<tgmath.h>` synopsis [depr.tgmath.h.syn]~~

~~`#include <cmath>`~~
~~`#include <complex>`~~

1. ~~The header `<tgmath.h>` behaves as if it simply includes the headers `<cmath>` (26.8.1) and `<complex>` (26.4.1).~~
2. ~~[*Note:* The overloads provided in C by type-generic macros are already provided in `<complex>` and `<cmath>` by "sufficient" additional overloads. — *end note*]~~
3. ~~[*Note:* Names introduced by `<cmath>` or `<complex>` in namespace std are not placed into the global namespace scope by `<tgmath.h>`. — *end note*]~~

~~D.9.6 Other C headers [depr.c.headers.other]~~

1. ~~Every C header other than `<complex.h>` (D.9.1), `<iso646.h>` (D.9.2), `<stdalign.h>` (D.9.3), `<stdbool.h>` (D.9.4), and `<tgmath.h>` (D.9.5), each of which has a name of the form `<name.h>`, behaves as if each name placed in the standard library namespace by the corresponding `<cname>` header is placed within the global namespace scope, except for the functions described in 26.8.6, the declaration of `std::byte` (17.2.1), and the functions and function templates described in 17.2.5. It is unspecified whether these names are first declared or defined within namespace scope (6.4.6) of the namespace `std` and are then injected into the global namespace scope by explicit using-declarations (9.9).~~
2. ~~[*Example:* The header `<cstdlib>` assuredly provides its declarations and definitions within the namespace `std`. It may also provide these names within the global namespace. The header `<stdlib.h>` assuredly provides the same declarations and definitions within the global namespace, much as in the C Standard. It may also provide these names within the namespace `std`. — *end example*]~~

C.6.1 Modifications to headers [diff.mods.to.headers]

1. For compatibility with the C standard library, the C++ standard library provides the C headers enumerated in ~~D.9, but their use is deprecated in C++~~ **[c.headers]**.
2. There are no C++ headers for the C headers `<complex.h>`, `<stdatomic.h>`, `<iso646.h>`, `<stdalign.h>`, `<stdbool.h>`, `<stdnoreturn.h>`, `<tgmath.h>`, and `<threads.h>`, nor are the C headers themselves part of C++.
3. ~~The headers `<ccomplex>` (29.5.11) and `<ctgmath>` (29.9.6), as well as their corresponding C headers `<complex.h>` and `<tgmath.h>`, do not contain any of the content from the C standard library and instead merely include other headers from the C++ standard library.~~
4. ~~The headers `<ciso646>`, `<cstdalign>` (21.10.4), and `<cstdbool>` (21.10.3) are meaningless in C++. Use of the C++ headers `<ccomplex>`, `<cstdalign>`, `<cstdbool>`, and `<ctgmath>` is deprecated (D.4).~~

Draft compatibility note for Annex C.

strong recommendation (B): Undeprecate the remaining **[depr.c.headers]** and move directly into 16.5.5.2 **[res.on.headers]**.

16.5.5.2 Headers [res.on.headers]

1. A C++ header may include other C++ headers. A C++ header shall provide the declarations and definitions that appear in its synopsis. A C++ header shown in its synopsis as including other C++ headers shall provide the declarations and definitions that appear in the synopses of those other headers.
2. Certain types and macros are defined in more than one header. Every such entity shall be defined such that any header that defines it may be included after any other header that also defines it (6.3).
3. ~~The C standard library headers (D.9) shall include only their corresponding C++ standard library header, as described in 16.5.1.2.~~

Actually, probably want to merge this into **16.5.1.2 Headers [headers]**. Also, fix up cross-references to list stable labels.

16.5.5.2.1 C headers [c.headers]

1. For compatibility with the C standard library, the C++ standard library provides the *C headers* shown in Table 147.

Table 147 — C headers [tab:depr.c.headers]

`<assert.h>`	`<inttypes.h>`	`<signal.h>`	`<stdio.h>`	`<wchar.h>`
`<complex.h>`	`<iso646.h>`	`<stdalign.h>`	`<stdlib.h>`	`<wctype.h>`
`<ctype.h>`	`<limits.h>`	`<stdarg.h>`	`<string.h>`	
`<errno.h>`	`<locale.h>`	`<stdbool.h>`	`<tgmath.h>`	
`<fenv.h>`	`<math.h>`	`<stddef.h>`	`<time.h>`	
`<float.h>`	`<setjmp.h>`	`<stdint.h>`	`<uchar.h>`	

16.5.5.2.1.1 Header `<complex.h>` synopsis [complex.h.syn]

```
#include <complex>
```

1. The header `<complex.h>` behaves as if it simply includes the header `<complex>` (26.4.1).
2. [*Note:* Names introduced by `<complex>` in namespace `std` are not placed into the global namespace scope by `<complex.h>`. *—end note*]

16.5.5.2.1.2 Header `<iso646.h>` synopsis [iso646.h.syn]

1. The C++ header `<iso646.h>` is empty. [*Note:* `and`, `and_eq`, `bitand`, `bitor`, `compl`, `not_eq`, `not`, `or`, `or_eq`, `xor`, and `xor_eq` are keywords in this International Standard (5.11). *—end note*]

16.5.5.2.1.3 Header `<stdalign.h>` synopsis [stdalign.h.syn]

```
#define __alignas_is_defined 1
```

1. The contents of the C++ header `<stdalign.h>` are the same as the C standard library header `<stdalign.h>`, with the following changes: The header `<stdalign.h>` does not define a macro named alignas.

   See also: ISO C 7.15

16.5.5.2.1.4 Header `<stdbool.h>` synopsis [stdbool.h.syn]

```
#define __bool_true_false_are_defined 1
```

1. The contents of the C++ header `<stdbool.h>` are the same as the C standard library header `<stdbool.h>`, with the following changes: The header `<stdbool.h>` does not define macros named `bool`, `true`, or `false`.

   See also: ISO C 7.18

16.5.5.2.1.5 Header `<tgmath.h>` synopsis [tgmath.h.syn]

```
#include <cmath>
#include <complex>
```

1. The header `<tgmath.h>` behaves as if it simply includes the headers `<cmath>` (26.8.1) and `<complex>` (26.4.1).
2. [*Note:* The overloads provided in C by type-generic macros are already provided in `<complex>` and `<cmath>` by "sufficient" additional overloads. *—end note*]
3. [*Note:* Names introduced by `<cmath>` or `<complex>` in namespace std are not placed into the global namespace scope by `<tgmath.h>`. *—end note*]

16.5.5.2.1.6 Other C headers [c.headers.other]

1. Every C header other than `<complex.h>` (D.9.1), `<iso646.h>` (D.9.2), `<stdalign.h>` (D.9.3), `<stdbool.h>` (D.9.4), and `<tgmath.h>` (D.9.5), each of which has a name of the form `<name.h>`, behaves as if each name placed in the standard library namespace by the corresponding `<cname>` header is placed within the global namespace scope, except for the functions described in 26.8.6, the declaration of `std::byte` (17.2.1), and the functions and function templates described in 17.2.5. It is unspecified whether these names are first declared or defined within namespace scope (6.4.6) of the namespace `std` and are then injected into the global namespace scope by explicit using-declarations (9.9).
2. [*Example:* The header `<cstdlib>` assuredly provides its declarations and definitions within the namespace `std`. It may also provide these names within the global namespace. The header `<stdlib.h>` assuredly provides the same

declarations and definitions within the global namespace, much as in the C Standard. It may also provide these names within the namespace `std`. —*end example*]

D.9 C headers [depr.c.headers]

1. For compatibility with the C standard library, the C++ standard library provides the *C headers* shown in Table 147.

Table 147 — C headers [tab:depr.c.headers]

`<assert.h>`	`<inttypes.h>`	`<signal.h>`	`<stdio.h>`	`<wchar.h>`
`<complex.h>`	`<iso646.h>`	`<stdalign.h>`	`<stdlib.h>`	`<wctype.h>`
`<ctype.h>`	`<limits.h>`	`<stdarg.h>`	`<string.h>`	
`<errno.h>`	`<locale.h>`	`<stdbool.h>`	`<tgmath.h>`	
`<fenv.h>`	`<math.h>`	`<stddef.h>`	`<time.h>`	
`<float.h>`	`<setjmp.h>`	`<stdint.h>`	`<uchar.h>`	

D.9.1 Header `<complex.h>` synopsis [depr.complex.h.syn]

```
#include <complex>
```

1. The header `<complex.h>` behaves as if it simply includes the header `<complex>` (26.4.1).
2. [*Note:* Names introduced by `<complex>` in namespace `std` are not placed into the global namespace scope by `<complex.h>`. —*end note*]

D.9.2 Header `<iso646.h>` synopsis [depr.iso646.h.syn]

1. The C++ header `<iso646.h>` is empty. [*Note:* `and`, `and_eq`, `bitand`, `bitor`, `compl`, `not_eq`, `not`, `or`, `or_eq`, `xor`, and `xor_eq` are keywords in this International Standard (5.11). —*end note*]

D.9.3 Header `<stdalign.h>` synopsis [depr.stdalign.h.syn]

```
#define __alignas_is_defined 1
```

1. The contents of the C++ header `<stdalign.h>` are the same as the C standard library header `<stdalign.h>`, with the following changes: The header `<stdalign.h>` does not define a macro named alignas.

   See also: ISO C 7.15

D.9.4 Header `<stdbool.h>` synopsis [depr.stdbool.h.syn]

```
#define __bool_true_false_are_defined 1
```

1. The contents of the C++ header `<stdbool.h>` are the same as the C standard library header `<stdbool.h>`, with the following changes: The header `<stdbool.h>` does not define macros named `bool`, `true`, or `false`.

   See also: ISO C 7.18

D.9.5 Header `<tgmath.h>` synopsis [depr.tgmath.h.syn]

```
#include <cmath>
#include <complex>
```

1. The header `<tgmath.h>` behaves as if it simply includes the headers `<cmath>` (26.8.1) and `<complex>` (26.4.1).
2. [*Note:* The overloads provided in C by type-generic macros are already provided in `<complex>` and `<cmath>` by "sufficient" additional overloads. —*end note*]
3. [*Note:* Names introduced by `<cmath>` or `<complex>` in namespace std are not placed into the global namespace scope by `<tgmath.h>`. —*end note*]

D.9.6 Other C headers [depr.c.headers.other]

1. Every C header other than `<complex.h>` (D.9.1), `<iso646.h>` (D.9.2), `<stdalign.h>` (D.9.3), `<stdbool.h>` (D.9.4), and `<tgmath.h>` (D.9.5), each of which has a name of the form `<name.h>`, behaves as if each name placed in the standard library namespace by the corresponding `<cname>` header is placed within the global namespace scope, except for the functions described in 26.8.6, the declaration of `std::byte` (17.2.1), and the functions and function templates described in 17.2.5. It is unspecified whether these names are first declared or defined within namespace scope (6.4.6) of the namespace `std` and are then injected into the global namespace scope by explicit using-declarations (9.9).
2. [*Example:* The header `<cstdlib>` assuredly provides its declarations and definitions within the namespace `std`. It may also provide these names within the global namespace. The header `<stdlib.h>` assuredly provides the same

declarations and definitions within the global namespace, much as in the C Standard. It may also provide these names within the namespace `std`. — *end example*]

Weak recommendation: Remove entirely from C++23.

D.9 C headers [depr.c.headers]

1. For compatibility with the C standard library, the C++ standard library provides the *C headers* shown in Table 147.

Table 147 — C headers [tab:depr.c.headers]

`<assert.h>`	`<inttypes.h>`	`<signal.h>`	`<stdio.h>`	`<wchar.h>`
`<complex.h>`	`<iso646.h>`	`<stdalign.h>`	`<stdlib.h>`	`<wctype.h>`
`<ctype.h>`	`<limits.h>`	`<stdarg.h>`	`<string.h>`	
`<errno.h>`	`<locale.h>`	`<stdbool.h>`	`<tgmath.h>`	
`<fenv.h>`	`<math.h>`	`<stddef.h>`	`<time.h>`	
`<float.h>`	`<setjmp.h>`	`<stdint.h>`	`<uchar.h>`	

D.9.1 Header `<complex.h>` synopsis [depr.complex.h.syn]

```
#include <complex>
```

1. The header `<complex.h>` behaves as if it simply includes the header `<complex>` (26.4.1).
2. [*Note:* Names introduced by `<complex>` in namespace `std` are not placed into the global namespace scope by `<complex.h>`. — *end note*]

D.9.2 Header `<iso646.h>` synopsis [depr.iso646.h.syn]

1. The C++ header `<iso646.h>` is empty. [*Note:* `and`, `and_eq`, `bitand`, `bitor`, `compl`, `not_eq`, `not`, `or`, `or_eq`, `xor`, and `xor_eq` are keywords in this International Standard (5.11). — *end note*]

D.9.3 Header `<stdalign.h>` synopsis [depr.stdalign.h.syn]

```
#define __alignas_is_defined 1
```

1. The contents of the C++ header `<stdalign.h>` are the same as the C standard library header `<stdalign.h>`, with the following changes: The header `<stdalign.h>` does not define a macro named alignas.

   See also: ISO C 7.15

D.9.4 Header `<stdbool.h>` synopsis [depr.stdbool.h.syn]

```
#define __bool_true_false_are_defined 1
```

1. The contents of the C++ header `<stdbool.h>` are the same as the C standard library header `<stdbool.h>`, with the following changes: The header `<stdbool.h>` does not define macros named `bool`, `true`, or `false`.

   See also: ISO C 7.18

D.9.5 Header `<tgmath.h>` synopsis [depr.tgmath.h.syn]

```
#include <cmath>
#include <complex>
```

1. The header `<tgmath.h>` behaves as if it simply includes the headers `<cmath>` (26.8.1) and `<complex>` (26.4.1).
2. [*Note:* The overloads provided in C by type-generic macros are already provided in `<complex>` and `<cmath>` by "sufficient" additional overloads. — *end note*]
3. [*Note:* Names introduced by `<cmath>` or `<complex>` in namespace std are not placed into the global namespace scope by `<tgmath.h>`. — *end note*]

D.9.6 Other C headers [depr.c.headers.other]

1. Every C header other than `<complex.h>` (D.9.1), `<iso646.h>` (D.9.2), `<stdalign.h>` (D.9.3), `<stdbool.h>` (D.9.4), and `<tgmath.h>` (D.9.5), each of which has a name of the form `<name.h>`, behaves as if each name placed in the standard library namespace by the corresponding `<cname>` header is placed within the global namespace scope, except for the functions described in 26.8.6, the declaration of `std::byte` (17.2.1), and the functions and function templates described

~~in 17.2.5. It is unspecified whether these names are first declared or defined within namespace scope (6.4.6) of the namespace `std` and are then injected into the global namespace scope by explicit using-declarations (9.9).~~

2. ~~[*Example:* The header `<cstdlib>` assuredly provides its declarations and definitions within the namespace `std`. It may also provide these names within the global namespace. The header `<stdlib.h>` assuredly provides the same declarations and definitions within the global namespace, much as in the C Standard. It may also provide these names within the namespace `std`. — *end example*]~~

Revise compatibility note for Annex C. This wording needs a little more thought.

C.6.1 Modifications to headers [diff.mods.to.headers]

1. For compatibility with the C standard library, the C++ standard library provides the C headers enumerated in ~~D.9, but their use is deprecated in C++~~**[c.headers]**.
2. ~~There are no C++ headers for the C headers `<stdatomic.h>`, `<stdnoreturn.h>`, and `<threads.h>`, nor are the C headers themselves part of C++.~~
3. ~~The headers `<ccomplex>` (29.5.11) and `<ctgmath>` (29.9.6), as well as their corresponding C headers `<complex.h>` and `<tgmath.h>`, do not contain any of the content from the C standard library and instead merely include other headers from the C++ standard library.~~
4. ~~The headers `<ciso646>`, `<cstdalign>` (21.10.4), and `<cstdbool>` (21.10.3) are meaningless in C++. Use of the C++ headers `<ccomplex>`, `<cstdalign>`, `<cstdbool>`, and `<ctgmath>` is deprecated (D.4).~~

LEWGI Review: To be determined...

D.10 Requires paragraph [depr.res.on.required]

First deprecated: C++20

This style of documentation was deprecated editorially following the application of a sequence of papers to update each main library clause, consistently following the new conventions established by paper P0788R3

Note that no review of this section will be needed if the options to remove all of D.11, D.13, D.18, D.21, and D.23 are accepted, as there would be nothing left to review.

As library clauses continue to be deprecated and migrate to Annex D, use of this deprecated convention will become internally less consistent, so the strong recommendation is to apply the new tags consistently through the parts of Annex D that are retained. The weak recommendation is to let this term remain until the last clause using it has been removed (or undeprecated and updated) in a future standard.

Strong recommendation: Remove this feature from C++23.

~~**D.10 Requires paragraph [depr.res.on.required]**~~

1. ~~In addition to the elements specified in 16.4.1.4, descriptions of function semantics may also contain a *Requires:* element to denote the preconditions for calling a function.~~
2. ~~Violation of any preconditions specified in a function's *Requires:* element results in undefined behavior unless the function's *Throws:* element specifies throwing an exception when the precondition is violated.~~

D.11 Relational operators [depr.relops]

> The *Cpp17OldConcept* requirements contain a semantic component, so the most appropriate replacement for a *Requires* clause is a *Preconditions* clause.

1. The header `<utility>` (20.2.1) has the following additions:

```
namespace std::rel_ops {
  template<class T> bool operator!=(const T&, const T&);
  template<class T> bool operator> (const T&, const T&);
  template<class T> bool operator<=(const T&, const T&);
  template<class T> bool operator>=(const T&, const T&);
}
```

2. To avoid redundant definitions of `operator!=` out of `operator==` and operators `>`, `<=`, and `>=` out of `operator<`, the library provides the following:

```
template<class T> bool operator!=(const T& x, const T& y);
```

3. ~~*Requires:* Type `T` is *Cpp17EqualityComparable* (Table 25).~~ *Preconditions:* `T` meets the *Cpp17EqualityComparable* requirements (Table 25).
4. *Returns:* `!(x == y)`.

```
template<class T> bool operator>(const T& x, const T& y);
```

5. ~~*Requires:* Type `T` is *Cpp17LessThanComparable* (Table 26).~~ *Preconditions:* `T` meets the *Cpp17LessThanComparable* requirements (Table 26).
6. *Returns:* `y < x`.

```
template<class T> bool operator<=(const T& x, const T& y);
```

7. ~~*Requires:* Type `T` is *Cpp17LessThanComparable* (Table 26).~~ *Preconditions:* `T` meets the *Cpp17LessThanComparable* requirements (Table 26).
8. *Returns:* `!(y < x)`.

```
template<class T> bool operator>=(const T& x, const T& y);
```

9. ~~*Requires:* Type `T` is *Cpp17LessThanComparable* (Table 26).~~ *Preconditions:* `T` meets the *Cpp17LessThanComparable* requirements (Table 26).
10. *Returns:* `!(x < y)`.

D.13 Deprecated type traits [depr.meta.types]

1. The header `<type_traits>` (20.15.2) has the following additions:

```
namespace std {
  template<class T> struct is_pod;
  template<class T> inline constexpr bool is_pod_v = is_pod<T>::value;
}
```

2. The behavior of a program that adds specializations for any of the templates defined in this subclause is undefined, unless explicitly permitted by the specification of the corresponding template.

```
template<class T> struct is_pod;
```

3. ~~*Requires*~~*Mandates*: `remove_all_extents_t<T>` shall be a complete type or *cv* `void`.
4. *Remarks:* `is_pod<T>` is a *Cpp17UnaryTypeTrait* (20.15.1) with a base characteristic of `true_type` if `T` is a POD type, and `false_type` otherwise. A POD class is a class that is both a trivial class and a standard-layout class, and has no non-static data members of type non-POD class (or array thereof). A POD type is a scalar type, a POD class, an array of such a type, or a cv-qualified version of one of these types.
5. [*Note:* It is unspecified whether a closure type (7.5.5.1) is a POD type. *—end note*]

D.18 Deprecated `shared_ptr` atomic access [depr.util.smartptr.shared.atomic]

1. The header `<memory>` (20.10.2) has the following additions:

```
namespace std {
template <class T>
  bool atomic_is_lock_free(const shared_ptr<T>* p);

template <class T>
  shared_ptr<T> atomic_load(const shared_ptr<T>* p);
template <class T>
  shared_ptr<T> atomic_load_explicit(const shared_ptr<T>* p, memory_order mo);

template <class T>
  void atomic_store(shared_ptr<T>* p, shared_ptr<T> r);
template <class T>
  void atomic_store_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);

template <class T>
  shared_ptr<T> atomic_exchange(shared_ptr<T>* p, shared_ptr<T> r);
template <class T>
  shared_ptr<T> atomic_exchange_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);

template <class T>
  bool atomic_compare_exchange_weak(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
template <class T>
  bool atomic_compare_exchange_strong(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
template <class T>
  bool atomic_compare_exchange_weak_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
template <class T>
```

```
  bool atomic_compare_exchange_strong_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
}
```

2. Concurrent access to a `shared_ptr` object from multiple threads does not introduce a data race if the access is done exclusively via the functions in this section and the instance is passed as their first argument.
3. The meaning of the arguments of type `memory_order` is explained in 31.4.

```
template<class T>
  bool atomic_is_lock_free(const shared_ptr<T>* p);
```

4. ~~*Requires*~~*Preconditions*: `p` ~~shall not be~~is not null.
5. *Returns:* `true` if atomic access to `*p` is lock-free, `false` otherwise.
6. *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_load(const shared_ptr<T>* p);
```

7. ~~*Requires*~~*Preconditions*: `p` ~~shall not be~~is not null.
8. *Returns:* `atomic_load_explicit(p, memory_order::seq_cst)`.
9. *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_load_explicit(const shared_ptr<T>* p, memory_order mo);
```

10. ~~*Requires*~~*Preconditions*: `p` ~~shall not be~~is not null.
11. ~~*Requires*~~*Preconditions*: `mo` ~~shall not be~~is neither `memory_order::release` nor `memory_order::acq_rel`.
12. *Returns:* `*p`.
13. *Throws:* Nothing.

```
template<class T>
  void atomic_store(shared_ptr<T>* p, shared_ptr<T> r);
```

14. ~~*Requires*~~*Preconditions*: `p` ~~shall not be~~is not null.
15. *Effects:* As if by `atomic_store_explicit(p, r, memory_order::seq_cst)`.
16. *Throws:* Nothing.

```
template<class T>
  void atomic_store_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);
```

17. ~~*Requires*~~*Preconditions*: `p` ~~shall not be~~is not null.
18. *Requires:* `mo` shall not be `memory_order::acquire` or `memory_order::acq_rel`.
19. *Effects:* As if by `p->swap(r)`.
20. *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_exchange(shared_ptr<T>* p, shared_ptr<T> r);
```

21. ~~*Requires*~~*Preconditions*: `p` ~~shall not be~~is not null.
22. *Returns:* `atomic_exchange_explicit(p, r, memory_order::seq_cst)`.
23. *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_exchange_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);
```

24. ~~*Requires*~~*Preconditions*: `p` ~~shall not be~~is not null.
25. *Effects:* As if by `p->swap(r)`.
26. *Returns:* The previous value of `*p`.
27. *Throws:* Nothing.

```
template<class T>
  bool atomic_compare_exchange_weak(shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
```

28. ~~*Requires*~~*Preconditions*: `p` ~~shall not be~~is not null.
29. *Returns:* `atomic_compare_exchange_weak_explicit(p, v, w, memory_order::seq_cst, memory_order::seq_cst)`.
30. *Throws:* Nothing.

```
template<class T>
  bool atomic_compare_exchange_strong(shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
```

31. *Returns:* `atomic_compare_exchange_strong_explicit(p, v, w, memory_order::seq_cst, memory_order::seq_cst)`.

```
template <class T>
  bool atomic_compare_exchange_weak_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
template <class T>
  bool atomic_compare_exchange_strong_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
```

32. ~~*Requires*~~*Preconditions*: `p` ~~shall not be~~is not and `v` ~~shall not be~~is not null. The `failure` argument ~~shall not be~~is neither `memory_order::release` nor `memory_order::acq_rel`.
33. *Effects:* If `*p` is equivalent to `*v`, assigns `w` to `*p` and has synchronization semantics corresponding to the value of `success`, otherwise assigns `*p` to `*v` and has synchronization semantics corresponding to the value of `failure`.
34. *Returns:* `true` if `*p` was equivalent to `*v`, `false` otherwise.
35. *Throws:* Nothing.
36. *Remarks:* Two `shared_ptr` objects are equivalent if they store the same pointer value and share ownership. The weak form may fail spuriously. See 31.8.1.

D.21 Deprecated convenience conversion interfaces [depr.conversions]

D.21.1 Class template wstring_convert [depr.conversions.string]

1. Class template `wstring_convert` performs conversions between a wide string and a byte string. It lets you specify a code conversion facet (like class template `codecvt`) to perform the conversions, without affecting any streams or locales. [*Example:* If you want to use the code conversion facet `codecvt_utf8` to output to `cout` a UTF-8 multibyte sequence corresponding to a wide string, but you don't want to alter the locale for `cout`, you can write something like:

```
std::wstring_convert<std::codecvt_utf8<wchar_t>> myconv;
std::string mbstring = myconv.to_bytes(L"Hello\n");
std::cout << mbstring;
```

— end example]

```
namespace std {
template <class Codecvt, class Elem = wchar_t,
          class WideAlloc = allocator<Elem>,
          class ByteAlloc = allocator<char>>
  class wstring_convert {
  public:
    using byte_string = basic_string<char, char_traits<char>, ByteAlloc>;
    using wide_string = basic_string<Elem, char_traits<Elem>, WideAlloc>;
    using state_type  = typename Codecvt::state_type;
    using int_type    = typename wide_string::traits_type::int_type;

    wstring_convert() : wstring_convert(new Codecvt) {}
    explicit wstring_convert(Codecvt* pcvt = new Codecvt);
    wstring_convert(Codecvt* pcvt, state_type state);
    explicit wstring_convert(const byte_string& byte_err,
                             const wide_string& wide_err = wide_string());
    ~wstring_convert();

    wstring_convert(const wstring_convert&) = delete;
    wstring_convert& operator=(const wstring_convert&) = delete;

    wide_string from_bytes(char byte);
    wide_string from_bytes(const char* ptr);
    wide_string from_bytes(const byte_string& str);
    wide_string from_bytes(const char* first, const char* last);

    byte_string to_bytes(Elem wchar);
    byte_string to_bytes(const Elem* wptr);
    byte_string to_bytes(const wide_string& wstr);
    byte_string to_bytes(const Elem* first, const Elem* last);

    size_t converted() const noexcept;
    state_type state() const;

  private:
    byte_string byte_err_string;  // Exposition only
    wide_string wide_err_string;  // Exposition only
    Codecvt* cvtptr;              // Exposition only
    state_type cvtstate;          // Exposition only
    size_t cvtcount;              // Exposition only
  };
}
```

2. The class template describes an object that controls conversions between wide string objects of class `basic_string<Elem, char_traits<Elem>, WideAlloc>` and byte string objects of class `basic_string<char, char_traits<char>, ByteAlloc>`. The class template defines the types `wide_string` and `byte_string` as synonyms for these two types. Conversion between a sequence of `Elem` values (stored in a `wide_string` object) and multibyte sequences (stored in a `byte_string` object) is performed by an object of class `Codecvt`, which meets the requirements of the standard code-conversion facet `codecvt<Elem, char, mbstate_t>`.
3. An object of this class template stores:
    1. — `byte_err_string` — a byte string to display on errors
    2. — `wide_err_string` — a wide string to display on errors
    3. — `cvtptr` — a pointer to the allocated conversion object (which is freed when the `wstring_convert` object is destroyed)
    4. — `cvtstate` — a conversion state object
    5. — `cvtcount` — a conversion count

```
using byte_string = basic_string<char, char_traits<char>, ByteAlloc>;
```

4. The type shall be a synonym for `basic_string<char, char_traits<char>, ByteAlloc>`

```
size_t converted() const noexcept;
```

5. *Returns:* `cvtcount`.

```
wide_string from_bytes(char byte);
wide_string from_bytes(const char* ptr);
wide_string from_bytes(const byte_string& str);
wide_string from_bytes(const char* first, const char* last);
```

6. *Effects:* The first member function shall convert the single-element sequence `byte` to a wide string. The second member function shall convert the null-terminated sequence beginning at `ptr` to a wide string. The third member function shall convert the sequence stored in str to a wide string. The fourth member function shall convert the sequence defined by the range `[first, last)` to a wide string.
7. In all cases:
    1. — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.
    2. — The number of input elements successfully converted shall be stored in `cvtcount`.
8. *Returns:* If no conversion error occurs, the member function shall return the converted wide string. Otherwise, if the object was constructed with a wide-error string, the member function shall return the wide-error string. Otherwise, the member function throws an object of class `range_error`.

```
using int_type = typename wide_string::traits_type::int_type;
```

9. The type shall be a synonym for `wide_string::traits_type::int_type`.

```
state_type state() const;
```

10. returns `cvtstate`.

```
using state_type = typename Codecvt::state_type;
```

11. The type shall be a synonym for `Codecvt::state_type`.

```
byte_string to_bytes(Elem wchar);
byte_string to_bytes(const Elem* wptr);
byte_string to_bytes(const wide_string& wstr);
byte_string to_bytes(const Elem* first, const Elem* last);
```

12. *Effects:* The first member function shall convert the single-element sequence `wchar` to a byte string. The second member function shall convert the null-terminated sequence beginning at `wptr` to a byte string. The third member function shall convert the sequence stored in wstr to a byte string. The fourth member function shall convert the sequence defined by the range `[first, last)` to a byte string.
13. In all cases:
    1. — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.
    2. — The number of input elements successfully converted shall be stored in `cvtcount`.
14. *Returns:* If no conversion error occurs, the member function shall return the converted byte string. Otherwise, if the object was constructed with a byte-error string, the member function shall return the byte-error string. Otherwise, the member function shall throw an object of class `range_error`.

```
using wide_string = basic_string<Elem, char_traits<Elem>, WideAlloc>;
```

15. The type shall be a synonym for `basic_string<Elem, char_traits<Elem>, WideAlloc>`.

```
explicit wstring_convert(Codecvt* pcvt = new Codecvt);
wstring_convert(Codecvt* pcvt, state_type state);
explicit wstring_convert(const byte_string& byte_err,
    const wide_string& wide_err = wide_string());
```

16. ~~*Requires*~~*Preconditions*: For the first and second constructors, `pcvt != nullptr`.
17. *Effects:* The first constructor shall store `pcvt` in `cvtptr` and default values in `cvtstate`, `byte_err_string`, and `wide_err_string`. The second constructor shall store `pcvt` in `cvtptr`, `state` in `cvtstate`, and default values in `byte_err_string` and `wide_err_string`; moreover the stored state shall be retained between calls to `from_bytes` and `to_bytes`. The third constructor shall store `new Codecvt` in `cvtptr`, `state_type()` in `cvtstate`, `byte_err` in `byte_err_string`, and `wide_err` in `wide_err_string`.

```
~wstring_convert();
```

18. *Effects:* The destructor shall delete `cvtptr`.

D.21.2 Class template wbuffer_convert [depr.conversions.buffer]

1. Class template `wbuffer_convert` looks like a wide stream buffer, but performs all its I/O through an underlying byte stream buffer that you specify when you construct it. Like class template `wstring_convert`, it lets you specify a code conversion facet to perform the conversions, without affecting any streams or locales.

```
namespace std {
template <class Codecvt, class Elem = wchar_t, class Tr = char_traits<Elem>>
  class wbuffer_convert
    : public basic_streambuf<Elem, Tr> {
  public:
    using state_type = typename Codecvt::state_type;

    wbuffer_convert() : wbuffer_convert(nullptr) {}
    explicit wbuffer_convert(streambuf* bytebuf = 0,
                             Codecvt* pcvt = new Codecvt,
                             state_type state = state_type());

    ~wbuffer_convert();

    wbuffer_convert(const wbuffer_convert&) = delete;
    wbuffer_convert& operator=(const wbuffer_convert&) = delete;

    streambuf* rdbuf() const;
    streambuf* rdbuf(streambuf* bytebuf);

    state_type state() const;

  private:
    streambuf* bufptr;        // exposition only
    Codecvt* cvtptr;          // exposition only
    state_type cvtstate;      // exposition only
  };
}
```

2. The class template describes a stream buffer that controls the transmission of elements of type `Elem`, whose character traits are described by the class `Tr`, to and from a byte stream buffer of type `streambuf`. Conversion between a sequence of `Elem` values and multibyte sequences is performed by an object of class `Codecvt`, which shall meet the requirements of the standard code-conversion facet `codecvt<Elem, char, mbstate_t>`.
3. An object of this class template stores:
    1. — `bufptr` — a pointer to its underlying byte stream buffer
    2. — `cvtptr` — a pointer to the allocated conversion object (which is freed when the `wbuffer_convert` object is destroyed)
    3. — `cvtstate` — a conversion state object

```
state_type state() const;
```

4. *Returns:* `cvtstate`.

```
streambuf* rdbuf() const;
```

5. *Returns:* `bufptr`.

```
streambuf* rdbuf(streambuf* bytebuf);
```

6. *Effects:* Stores `bytebuf` in `bufptr`.
7. *Returns:* The previous value of `bufptr`.

```
using state_type = typename Codecvt::state_type;
```

8. The type shall be a synonym for `Codecvt::state_type`.

```
explicit wbuffer_convert(streambuf* bytebuf = 0,
    Codecvt* pcvt = new Codecvt, state_type state = state_type());
```

9. ~~*Requires*~~*Preconditions*: `pcvt != nullptr`.
10. *Effects:* The constructor constructs a stream buffer object, initializes `bufptr` to `bytebuf`, initializes `cvtptr` to `pcvt`, and initializes `cvtstate` to `state`.

```
~wbuffer_convert();
```

11. *Effects:* The destructor shall delete `cvtptr`.

D.23 Deprecated filesystem path factory functions [depr.fs.path.factory]

The *Requires* clause here covers both compile-time and run-time conditions, so is split into separate *Mandates* and *Preconditions* clauses. A drive-by fix further catches that several typedefs intending to refer to members of `path` are actually meaningless for a non-member factory function. The current form of wording was copy/pasted from the specification of constructors where the `path::` would be implicit.

1. The header `<filesystem>` (29.11.5) has the following additions:

```
template<class Source>
  path u8path(const Source& source);
template<class InputIterator>
  path u8path(InputIterator first, InputIterator last);
```

2. ~~*Requires:* The `source` and `[first, last)` sequences are UTF-8 encoded. The value type of `Source` and `InputIterator` is `char` or `char8_t`. `Source` meets the requirements specified in 29.11.7.3.~~
3. *Mandates:* The value type of `Source` and `InputIterator` is `char` or `char8_t`.
4. *Preconditions:* The `source` and `[first, last)` sequences are UTF-8 encoded. `Source` meets the requirements specified in 29.11.7.3.
5. *Returns:*
    1. — If `path::value_type` is `char` and the current native narrow encoding (29.11.7.2.2) is UTF-8, return `path(source)` or `path(first, last)`; otherwise,
    2. — if `path::value_type` is `wchar_t` and the native wide encoding is UTF-16, or if `path::value_type` is `char16_t` or `char32_t`, convert `source` or `[first,last)` to a temporary, `tmp`, of type `path::string_type` and return `path(tmp)`; otherwise,
    3. — convert `source` or `[first, last)` to a temporary, `tmp`, of type `u32string` and return `path(tmp)`.
6. *Remarks:* Argument format conversion (29.11.7.2.1) applies to the arguments for these functions. How Unicode encoding conversions are performed is unspecified.
7. [*Example:* A string is to be read from a database that is encoded in UTF-8, and used to create a directory using the native encoding for filenames:

```
namespace fs = std::filesystem;
std::string utf8_string = read_utf8_data();
fs::create_directory(fs::u8path(utf8_string));
```

For POSIX-based operating systems with the native narrow encoding set to UTF-8, no encoding or type conversion occurs.

For POSIX-based operating systems with the native narrow encoding not set to UTF-8, a conversion to UTF-32 occurs, followed by a conversion to the current native narrow encoding. Some Unicode characters may have no native character set representation.

For Windows-based operating systems a conversion from UTF-8 to UTF-16 occurs. —*end example*] [*Note:* The example above is representative of a historical use of `filesystem::u8path`. Passing a `std::u8string` to `path`'s constructor is preferred for an indication of UTF-8 encoding more consistent with `path`'s handling of other encodings. —*end note*]

Weak recommendation: take no action.

No change to draft.

LEWGI Review: To be determined...

D.11 Relational operators [depr.relops]

First deprecated: C++20

This feature was deprecated with the introduction of support for the 3-way comparison "spaceship" operator, by paper P0768R1.

The `std::rel_ops` namespace was introduced in the original C++ standard, so its removal would potentially impact on code written against C++98 and later standards. However, this paper will still recommend strongly for its removal from the C++23 standard. The feature relies on the antipattern of `using` a namespace in a namespace scope in order to provide comparison operators for my own type, typically in a header. Such usage is not recommended as the `using` will have an impact on code in other headers, and potentially introduce header order dependencies. A more targeted use within a function may serve as an adaption for 3rd party libraries, but again, by providing these operators for all types whether desired or not. As there are no viable base classes in this namespace, it was never possible to arrange for these operators to be discovered by ADL, so there was no better workaround to tame these functions.

The better solution appeared in C++20 with the introduction of the spaceship operator, and rules for synthesizing comparisons from a few primitive operators now built into the language. The typical remedy for much code will be to simply remove the offending `using namespace std::rel_ops;` from their code, and it should continue to work. As added insurance, the name `rel_ops` will be added to the list of zombie names reserved for previous standardization so that standard library vendors can continue to offer these as a compatibility shim for their customers for as long as they wish, so the impact on the user community of removing the specification of this feature from the standard is expected to be minimal.

The weak recommendation is that this feature was deprecated just one standard prior, so the user community may take a while to catch up and remove the dependencies from their code. Holding the feature in Annex D does little harm, but the wording should be updated to the latest library documentation style for C++23. We do not see any reason to consider undeprecation of this feature.

Strong recommendation: Remove this feature from C++23.

16.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization:
    1. — `auto_ptr`,
    2. — `auto_ptr_ref`,
    3. — `binary_function`,
    4. — `binary_negate`,
    5. — `bind1st`,
    6. — `bind2nd`,
    7. — `binder1st`,
    8. — `binder2nd`,
    9. — `const_mem_fun1_ref_t`,
    10. — `const_mem_fun1_t`,
    11. — `const_mem_fun_ref_t`,
    12. — `const_mem_fun_t`,
    13. — `get_temporary_buffer`,
    14. — `get_unexpected`,
    15. — `gets`,
    16. — `is_literal_type`,
    17. — `is_literal_type_v`,
    18. — `mem_fun1_ref_t`,
    19. — `mem_fun1_t`,
    20. — `mem_fun_ref_t`,
    21. — `mem_fun_ref`,
    22. — `mem_fun_t`,
    23. — `mem_fun`,
    24. — `not1`,
    25. — `not2`,
    26. — `pointer_to_binary_function`,
    27. — `pointer_to_unary_function`,
    28. — `ptr_fun`,
    29. — `random_shuffle`,
    30. — `raw_storage_iterator`,
    31. — `rel_ops`,
    32. — `result_of`,
    33. — `result_of_t`,
    34. — `return_temporary_buffer`,
    35. — `set_unexpected`,

36. — `unary_function`,
37. — `unary_negate`,
38. — `uncaught_exception`,
39. — `unexpected`, and
40. — `unexpected_handler`.

~~D.11 Relational operators [depr.relops]~~

1. ~~The header `<utility>` (20.2.1) has the following additions:~~

```
>namespace std::rel_ops {
  template<class T> bool operator!=(const T&, const T&);
  template<class T> bool operator> (const T&, const T&);
  template<class T> bool operator<=(const T&, const T&);
  template<class T> bool operator>=(const T&, const T&);
>}
```

2. ~~To avoid redundant definitions of `operator!=` out of `operator==` and operators `>`, `<=`, and `>=` out of `operator<`, the library provides the following:~~

```
template<class T> bool operator!=(const T& x, const T& y);
```

3. ~~*Requires:* Type `T` is *Cpp17EqualityComparable* (Table 23).~~
4. ~~*Returns:* `!(x == y)`.~~

```
template<class T> bool operator>(const T& x, const T& y);
```

5. ~~*Requires:* Type `T` is *Cpp17LessThanComparable* (Table 24).~~
6. ~~*Returns:* `y < x`.~~

```
template<class T> bool operator<=(const T& x, const T& y);
```

7. ~~*Requires:* Type `T` is *Cpp17LessThanComparable* (Table 24).~~
8. ~~*Returns:* `!(y < x)`.~~

```
template<class T> bool operator>=(const T& x, const T& y);
```

9. ~~*Requires:* Type `T` is *Cpp17LessThanComparable* (Table 24).~~
10. ~~*Returns:* `!(x < y)`.~~

Draft compatibility note for Annex C.

Weak recommendation: Clean up wording.

Update wording to replace *Requires* clauses (see D.10 Requires paragraph).

LEWGI Review: To be determined...

D.12 `char*` streams [depr.str.strstreams]

First deprecated: C++98

The `char*` streams were provided, pre-deprecated, in C++98 and have been considered for removal before. The underlying principle of not removing them until a suitable replacement is available still holds, so there should be nothing further to discuss at this point.

However, it should be noted that the `spanstream` library passed LEWG review for C++20, but ran out of LWG working time to integrate into the final standard. This library is expected to be the replacement facility we point users to in the future, so it may be reasonable to again consider the classic strstreams facility for removal in C++26.

There remains the alternative position that this facility has been a supported shipping part of the C++ standard for around 25 years when C++23 ships. If we have not made serious moves to remove the library in all that time, maybe we should consider undeprecating, and taking away the shadow of doubt over any code that reaches for this facility today.

Strong recommendation: take no action.

No change to draft.

Weak recommendation: Undeprecate the `char*` streams.

Wording to follow on demand, moving subclause D.6 into clause 29.

LEWGI Review: To be determined...

D.13 Deprecated type traits [depr.meta.types]

First deprecated: C++20

While traits have been deprecated in earlier versions of the standard, C++20 removed all traits deprecated in C++17 and earlier, so the traits that remain were all deprecated by C++20.

The `is_pod` trait was deprecated by paper P0767R1 as part of removing the POD vocabulary from the C++ standard, both core and library. The term had changed meaning so frequently that it no longer served as useful vocabulary. The type trait was extracted to Annex D, and now itself provides the only definition in the standard for a POD. Client code is encouraged to use the more specific traits for trivial and standard layout types to better describe their need.

The `is_pod` trait was first supplied as part of C++11, so its removal would potentially impact programs written against the C++11 standard or later. Users have had up to three years of implementations warning on use of the deprecated trait, so we could consider removal, with the usual proviso that the name is preserved as a zombie for previous standardization. As the case for removal is not urgent, the weak recommendation of this paper is to remove the trait from C++23.

However, the current wording does not follow library best practices, and should be updated to better specify the *Requires* clauses with our modern vocabulary. The strong recommendation is to retain it as deprecated in Annex D and revise the wording accordingly.

Strong recommendation: Clean up wording.

Update wording to replace *Requires* clauses (see D.10 Requires paragraph).

Weak recommendation: Remove the traits that can live on as zombies.

16.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization:
    1. — `auto_ptr`,
    2. — `auto_ptr_ref`,
    3. — `binary_function`,
    4. — `binary_negate`,
    5. — `bind1st`,
    6. — `bind2nd`,
    7. — `binder1st`,
    8. — `binder2nd`,
    9. — `const_mem_fun1_ref_t`,
    10. — `const_mem_fun1_t`,
    11. — `const_mem_fun_ref_t`,
    12. — `const_mem_fun_t`,
    13. — `get_temporary_buffer`,
    14. — `get_unexpected`,
    15. — `gets`,
    16. — `is_literal_type`,
    17. — `is_literal_type_v`,
    18. — `is_pod`,
    19. — `is_pod_v`,
    20. — `mem_fun1_ref_t`,
    21. — `mem_fun1_t`,
    22. — `mem_fun_ref_t`,
    23. — `mem_fun_ref`,
    24. — `mem_fun_t`,
    25. — `mem_fun`,
    26. — `not1`,
    27. — `not2`,
    28. — `pointer_to_binary_function`,
    29. — `pointer_to_unary_function`,
    30. — `ptr_fun`,
    31. — `random_shuffle`,
    32. — `raw_storage_iterator`,
    33. — `result_of`,
    34. — `result_of_t`,
    35. — `return_temporary_buffer`,

36. — `set_unexpected`,
37. — `unary_function`,
38. — `unary_negate`,
39. — `uncaught_exception`,
40. — `unexpected`, and
41. — `unexpected_handler`.

~~D.13 Deprecated Type Traits [depr.meta.type]~~

1. ~~The header `<type_traits>` (20.15.2) has the following additions:~~

   ```
   namespace std {
     template<class T> struct is_pod;
     template<class T> inline constexpr bool is_pod_v = is_pod<T>::value;
   }
   ```

2. ~~The behavior of a program that adds specializations for any of the templates defined in this subclause is undefined, unless explicitly permitted by the specification of the corresponding template.~~

   ```
   template<class T> struct is_pod;
   ```

3. ~~*Requires:* `remove_all_extents_t<T>` shall be a complete type or *cv* `void`.~~
4. ~~`is_pod<T>` is a *Cpp17UnaryTypeTrait* (20.15.1) with a base characteristic of `true_type` if `T` is a POD type, and `false_type` otherwise. A POD class is a class that is both a trivial class and a standard-layout class, and has no non-static data members of type non-POD class (or array thereof). A POD type is a scalar type, a POD class, an array of such a type, or a cv-qualified version of one of these types.~~
5. ~~[*Note:* It is unspecified whether a closure type (7.5.5.1) is a POD type. — *end note*]~~

Draft compatibility note for Annex C.

LEWGI Review: To be determined...

There is a preference for removing replaced facilities from the standard at the earliest opportunity, and letting the vendors remove the zombie implementations at a time of their own choosing, assessing their own customer demand.

D.14 Tuple [depr.tuple]

First deprecated: C++20

This feature was deprecated as part of the effort to cleanly introduce modules into the language, and was adopted by paper P1831R1. It potentially impacts on code written against C++11 and later standards.

This feature was deprecated only at the final meeting of the C++20 development cycle, and at the time this paper is written, there is no experience with how much code has been impacted by this deprecation. As such, it is too early to make any recommendation for a change with respect to this feature.

It is further noted that there is a coupling between the deprecated tuple traits API, and the deprecated support for volatile structured bindings (D.5). First, note that `volatile tuple` itself does not support structured bindings (nor do any `pair` and `array`) as there are no overloads for the `get` function taking references to `volatile` qualified objects. However, it is still possible for a user to customize their own type to support such `get` calls. If we remove the `volatile` support from the tuple traits by default, then the user would have to provide their own specializations for `tuple_size<volatile TYPE>` and `tuple_element<volatile TYPE>`, and similarly for the `const volatile` qualifier. Alternatively, we could ensure we remove the deprecated support for `volatile` structured bindings at the same time that we remove the tuple traits `volatile` API.

Strong recommendation: take no action.

No change to draft.

Weak recommendation: take no action.

No change to draft.

LEWGI Review: To be determined...

D.15 Variant [depr.variant]

First deprecated: C++20

This feature was deprecated as part of the effort to cleanly introduce modules into the language, and was adopted by paper P1831R1. It potentially impacts on code written against C++17 and later standards.

This feature was deprecated only at the final meeting of the C++20 development cycle, and at the time this paper is written, there is no experience with how much code has been impacted by this deprecation. However, `variant` has been in the language for a much shorter time than the similarly impacted `tuple`, and so it is likely that much less code would be impacted by its removal. Secondly, as `variant` has no `volatile` qualified member functions, nor external accessors like `get` accepting `volatile` variants, the scope for reasonable use of a `volatile` variant is vanishingly small. Therefore, the strong recommendation of this paper is to remove directly from C++23, and the weak recommendation is to hold it over for one more standard cycle, allowing more time for any vestigial usage to be reworked.

Strong recommendation: remove this feature from C++23.

~~**D.15 Variant [depr.variant]**~~

1. ~~The header `<variant>` (20.7.2) has the following additions:~~

```
namespace std {
  template<class T> struct variant_size<volatile T>;
  template<class T> struct variant_size<const volatile T>;
  template<size_t I, class T> struct variant_alternative<I, volatile T>;
  template<size_t I, class T> struct variant_alternative<I, const volatile T>;
}

template<class T> class variant_size<volatile T>;
template<class T> class variant_size<const volatile T>;
```

2. ~~Let `VS` denote `variant_size<T>` of the cv-unqualified type `T`. Then specializations of each of the two templates meet the *Cpp17UnaryTypeTrait* requirements with a base characteristic of `integral_constant<size_t, VS::value>`.~~

```
template<size_t I, class T> class variant_alternative<I, volatile T>;
template<size_t I, class T> class variant_alternative<I, const volatile T>;
```

3. ~~Let `VA` denote `variant_alternative<I, T>` of the cv-unqualified type `T`. Then specializations of each of the two templates meet the *Cpp17TransformationTrait* requirements with a member typedef type that names the following type:~~
   1. ~~— for the first specialization, `add_volatile_t<VA::type>`, and~~
   2. ~~— for the second specialization, `add_cv_t<VA::type>`.~~

Draft compatibility note for Annex C.

Weak recommendation: take no action.

No change to draft.

LEWGI Review: To be determined...

D.16 Deprecated iterator primitives [depr.iterator.primitives]

First deprecated: C++17

The class template `iterator` was first deprecated in C++17 by the paper P0174R2. The concern was that providing the needed support for iterator typenames through a templated base class, determining which name maps to which type purely by parameter order, was less clear than simply providing the needed names. Further, there were corner cases in usage that fell out of template syntax that made this tool hard to recommend as a simpler way of providing the type names, yet that was its whole reason to exist.

When this facility was reviewed for removal in C++20, it was noted that there were valid uses that relied on the default template arguments to deduce at least a few of the needed type names. Subsequent work on iterators and ranges in C++20 now means that work is also done by the primary `iterator_traits` template, and so the remaining use case (for new code) is also covered, making this class template strictly redundant.

The main concern that remains is breaking old code by removing this code from the standard libraries. That risk is ameliorated by the zombie names clause in the standard, allowing vendors to maintain their own support for as long as their customers demand. By the time C++23 ships, those customers will already have been on 6 years notice that their code might not be supported in future standards. However, we note the repeated use of the name `iterator` as a type within many containers means we might choose to leave this name off the zombie list. We conservatively place it there anyway, to ensure that we are covered by the previous standardization terminology to encompass uses other than as a container iterator typedef.

The strong recommendation of this paper is to remove this feature from the pending standard. The weak recommendation is to ask what further changes we would like to see before we could remove this feature. If there is no anticipated change of circumstance that would allow the standard to stop tracking this feature, then the recommendation should be to undeprecate, and restore this text to the main body of the standard.

Strong recommendation: remove this feature from C++23.

16.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization:
   1. — `auto_ptr`,
   2. — `auto_ptr_ref`,
   3. — `binary_function`,
   4. — `binary_negate`,
   5. — `bind1st`,
   6. — `bind2nd`,
   7. — `binder1st`,
   8. — `binder2nd`,
   9. — `const_mem_fun1_ref_t`,
   10. — `const_mem_fun1_t`,
   11. — `const_mem_fun_ref_t`,
   12. — `const_mem_fun_t`,
   13. — `get_temporary_buffer`,
   14. — `get_unexpected`,
   15. — `gets`,
   16. — `is_literal_type`,
   17. — `is_literal_type_v`,
   18. — `iterator`,
   19. — `mem_fun1_ref_t`,
   20. — `mem_fun1_t`,
   21. — `mem_fun_ref_t`,
   22. — `mem_fun_ref`,
   23. — `mem_fun_t`,
   24. — `mem_fun`,
   25. — `not1`,
   26. — `not2`,
   27. — `pointer_to_binary_function`,
   28. — `pointer_to_unary_function`,
   29. — `ptr_fun`,
   30. — `random_shuffle`,
   31. — `raw_storage_iterator`,
   32. — `result_of`,
   33. — `result_of_t`,
   34. — `return_temporary_buffer`,
   35. — `set_unexpected`,
   36. — `unary_function`,
   37. — `unary_negate`,
   38. — `uncaught_exception`,
   39. — `unexpected`, and
   40. — `unexpected_handler`.

D.16 Deprecated iterator primitives [depr.iterator.primitives]

D.16.1 Basic iterator [depr.iterator.basic]

1. The header <iterator> (23.2) has the following addition:

```
namespace std {
  template<class Category, class T, class Distance = ptrdiff_t,
    class Pointer = T*, class Reference = T&>
  struct iterator {
    using iterator_category = Category;
    using value_type        = T;
    using difference_type   = Distance;
    using pointer           = Pointer;
    using reference         = Reference;
  };
}
```

2. The `iterator` template may be used as a base class to ease the definition of required types for new iterators.
3. [*Note:* If the new iterator type is a class template, then these aliases will not be visible from within the iterator class's template definition, but only to callers of that class — *end note*]
4. [*Example:* If a C++ program wants to define a bidirectional iterator for some data structure containing `double` and such that it works on a large memory model of the implementation, it can do so with:

```
class MyIterator :
  public iterator<bidirectional_iterator_tag, double, long, T*, T&> {
  // code implementing ++, etc.
};

— end example ]
```

Draft compatibility note for Annex C.

Weak recommendation: Undeprecate the facility, and restore it to clause 23.

Wording to be supplied.

LEWGI Review: To be determined...

D.17 Deprecated `move_iterator` access [depr.move.iter.elem]

First deprecated: C++20

This feature was deprecated for C++20 by the paper P1252R2 highlighting the concern that for a move iterator adapter, intending to expose its target as an rvalue (or xvalue), the arrow operator must return the original adapted iterator, which will likely produce an lvalue when dereferenced. The operator is not fit for purpose, and cannot be fixed. The workaround for users is to dereference the move iterator with `operator *` and call the member they wish to access using the familiar `.` notation. This preserves the value category of the iterator's target.

The proposal for C++20 was to deprecate this operator, with a view to removal at a later date. While it may seem early, this is the first such later date appropriate to consider that removal.

Lacking clear evidence that this issue is causing widespread bugs in practice, the strong recommendation for this paper is to hold the feature in Annex D for another standard cycle, and strongly consider its removal in C++23. The weak recommendation is that with three years (or more) experience of deprecation warnings, we have given users enough notice, and it is time to remove this misleading feature now, from C++23. Code written against the C++11 standard or later might be impacted by this change.

Strong recommendation: Take no action.

No change to draft.

Weak recommendation: Remove this feature from C++23

D.17 Deprecated move_iterator access [depr.move.iter.elem]

1. The following member is declared in addition to those members specified in 23.5.3.5:

```
namespace std {
  template<class Iterator>
    class move_iterator {
    public:
      constexpr pointer operator->() const;
  };
}

constexpr pointer operator->() const;
```

2. *Returns:* `current`.

Draft compatibility note for Annex C.

LEWGI Review: To be determined...

D.18 Deprecated `shared_ptr` atomic access [depr.util.smartptr.shared.atomic]

First deprecated: C++20

The legacy C-style atomic API for manipulating shared pointers provided in C++11 is subtle, frequently misunderstood: a `shared_ptr` that is to be used with the atomic API can never be used directly, but can only be manipulated through the atomic API (other than construction and destruction). Its failure mode on misuse is silent undefined behavior, typically a data race.

C++20 provides a type-safe alternative that also provides support for `atomic<weak_ptr<T>>`.

Strong recommendation: Remove this feature at the earliest opportunity, ideally C++23.

D.18 Deprecated `shared_ptr` atomic access [depr.util.smartptr.shared.atomic]

1. The header `<memory>` (20.10.2) has the following additions:

```
namespace std {
template <class T>
  bool atomic_is_lock_free(const shared_ptr<T>* p);

template <class T>
  shared_ptr<T> atomic_load(const shared_ptr<T>* p);
template <class T>
  shared_ptr<T> atomic_load_explicit(const shared_ptr<T>* p, memory_order mo);

template <class T>
  void atomic_store(shared_ptr<T>* p, shared_ptr<T> r);
template <class T>
  void atomic_store_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);

template <class T>
  shared_ptr<T> atomic_exchange(shared_ptr<T>* p, shared_ptr<T> r);
template <class T>
  shared_ptr<T> atomic_exchange_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);

template <class T>
  bool atomic_compare_exchange_weak(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
template <class T>
  bool atomic_compare_exchange_strong(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
template <class T>
  bool atomic_compare_exchange_weak_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
template <class T>
  bool atomic_compare_exchange_strong_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
}
```

2. Concurrent access to a `shared_ptr` object from multiple threads does not introduce a data race if the access is done exclusively via the functions in this section and the instance is passed as their first argument.
3. The meaning of the arguments of type `memory_order` is explained in 31.4.

```
template<class T>
  bool atomic_is_lock_free(const shared_ptr<T>* p);
```

4. *Requires:* `p` shall not be null.
5. *Returns:* `true` if atomic access to `*p` is lock-free, `false` otherwise.
6. *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_load(const shared_ptr<T>* p);
```

7. *Requires:* `p` shall not be null.
8. *Returns:* `atomic_load_explicit(p, memory_order::seq_cst)`.
9. *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_load_explicit(const shared_ptr<T>* p, memory_order mo);
```

10. *Requires:* `p` shall not be null.
11. *Requires:* `mo` shall not be `memory_order::release` or `memory_order::acq_rel`.
12. *Returns:* `*p`.
13. *Throws:* Nothing.

```
template<class T>
  void atomic_store(shared_ptr<T>* p, shared_ptr<T> r);
```

14. *Requires:* `p` shall not be null.
15. *Effects:* As if by `atomic_store_explicit(p, r, memory_order::seq_cst)`.
16. *Throws:* Nothing.

```
template<class T>
  void atomic_store_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);
```

17. *Requires:* p shall not be null.
18. *Requires:* mo shall not be memory_order::acquire or memory_order::acq_rel.
19. *Effects:* As if by p->swap(r).
20. *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_exchange(shared_ptr<T>* p, shared_ptr<T> r);
```

21. *Requires:* p shall not be null.
22. *Returns:* atomic_exchange_explicit(p, r, memory_order::seq_cst).
23. *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_exchange_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);
```

24. *Requires:* p shall not be null.
25. *Effects:* As if by p->swap(r).
26. *Returns:* The previous value of *p.
27. *Throws:* Nothing.

```
template<class T>
  bool atomic_compare_exchange_weak(shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
```

28. *Requires:* p shall not be null.
29. *Returns:* atomic_compare_exchange_weak_explicit(p, v, w, memory_order::seq_cst, memory_order::seq_cst).
30. *Throws:* Nothing.

```
template<class T>
  bool atomic_compare_exchange_strong(shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
```

31. *Returns:* atomic_compare_exchange_strong_explicit(p, v, w, memory_order::seq_cst, memory_order::seq_cst).

```
template <class T>
  bool atomic_compare_exchange_weak_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
template <class T>
  bool atomic_compare_exchange_strong_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
```

32. *Requires:* p shall not be null and v shall not be null. The failure argument shall not be memory_order::release nor memory_order::acq_rel.
33. *Effects:* If *p is equivalent to *v, assigns w to *p and has synchronization semantics corresponding to the value of success, otherwise assigns *p to *v and has synchronization semantics corresponding to the value of failure.
34. *Returns:* true if *p was equivalent to *v, false otherwise.
35. *Throws:* Nothing.
36. *Remarks:* Two shared_ptr objects are equivalent if they store the same pointer value and share ownership. The weak form may fail spuriously. See 31.8.1.

Draft compatibility note for Annex C.

Weak recommendation: Remove this feature at the earliest opportunity *after* C++23.

Update wording to replace *Requires* clauses (see D.10 Requires paragraph).

LEWGI Review: To be determined...

D.19 Deprecated `basic_string` capacity [depr.string.capacity]

First deprecated: C++20

This feature was first deprecated for C++20 by the paper P0966R1. The deprecation was a consequence of cleaning up the behavior of the `reserve` function to no longer optionally reallocate on a request to shrink. The original C++98 specification for `basic_string` supplied a default argument of 0 for `reserve`, turning a call to `reserve()` into a non-binding `shrink_to_fit` request. Note that `shrink_to_fit` was added in C++11 to better support this use case. With the removal of the potentially reallocating behavior, `reserve()` is now a redundant function overload that is guaranteed to do nothing. Hence it was deprecated in C++20, with a view to removing it entirely in a later standard to eliminate on more legacy source of confusion from the standard.

As the feature was deprecated so recently, the strong recommendation of this paper is to make no changes for C++23, but strongly consider removal when it is time to review again for C++26. The weak recommendation is that this feature is small and obscure enough that it is better to remove now from C++23 than preserve for another three years into C++26.

Strong recommendation: Take no action.

No change to draft.

Weak recommendation: Remove this feature from C++23

~~**D.19 Deprecated basic_string capacity [depr.string.capacity]**~~

1. ~~The following member is declared in addition to those members specified in 21.3.2.4:~~

```
namespace std {
  template<class charT, class traits = char_traits<charT>,
           class Allocator = allocator<charT>>
    class basic_string {
    public:
      void reserve();
    };
}

void reserve();
```

2. ~~*Effects:* After this call, `capacity()` has an unspecified value greater than or equal to `size()`. [*Note:* This is a non-binding shrink to fit request. — *end note*]~~

Draft compatibility note for Annex C.

LEWGI Review: To be determined...

D.20 Deprecated Standard code conversion facets [depr.locale.stdcvt]

First deprecated: C++17

This feature was originally proposed for C++11 by paper N2007 and deprecated for C++17 by paper P0618R0. As noted at the time, the feature was underspecified and would require more work than we wished to invest to bring it up to standard. Since then SG16 has been convened and is producing a steady stream of work to bring reliable well-specified Unicode support to C++.

It should also be noted that this deprecated clause pins a dated reference to a 20 year old ISO standard (revised repeatedly over the intervening decades) purely to provide a definition of the term UCS2.

Given vendors propensity to provide ongoing support for these names under the zombie name reservations, the strong recommendation of this paper is to remove this library immediately from C++23, along with its binding reference to an obsolete Unicode standard. The weak recommendation is to do nothing at this point, until SG16 (or some other entity) produces a clean replacement for this facility.

We note that this feature was originally added at the Kona 2007 meeting so that (while motivated by likely user applications) the example in the then recently added [lib.conversions] called on standard rather than user-provided classes to illustrate use (adopted just one meeting earlier). Therefore, if we remove this library unilaterally, we should also revert that example back to its original spelling.

Strong recommendation: Remove this facility from the standard at the earliest opportunity.

2 Normative references [intro.refs]

1. The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.
    1. — Ecma International, ECMAScript Language Specification, Standard Ecma-262, third edition, 1999.
    2. — ISO/IEC 2382 (all parts), Information technology — Vocabulary
    3. — ISO 8601:2004, Data elements and interchange formats — Information interchange — Representation of dates and times
    4. — ISO/IEC 9899:2018, Programming languages — C
    5. — ISO/IEC 9945:2003, Information Technology — Portable Operating System Interface (POSIX)
    6. — ISO/IEC 10646, Information technology — Universal Coded Character Set (UCS)
    7. ~~— ISO/IEC 10646-1:1993, Information technology — Universal Multiple-Octet Coded Character Set (UCS) — Part 1: Architecture and Basic Multilingual Plane~~

8. — ISO/IEC/IEEE 60559:2011, Information technology — Microprocessor Systems — Floating-Point arithmetic
9. — ISO 80000-2:2009, Quantities and units — Part 2: Mathematical signs and symbols to be used in the natural sciences and technology

2. The library described in Clause 7 of ISO/IEC 9899:2018 is hereinafter called the C standard library.[1]
3. The operating system interface described in ISO/IEC 9945:2003 is hereinafter called POSIX.
4. The ECMAScript Language Specification described in Standard Ecma-262 is hereinafter called ECMA-262.
5. ~~[Note: References to ISO/IEC 10646-1:1993 are used only to support deprecated features (D.19). — end note]~~

16.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization:
   1. — `auto_ptr`,
   2. — `auto_ptr_ref`,
   3. — `binary_function`,
   4. — `binary_negate`,
   5. — `bind1st`,
   6. — `bind2nd`,
   7. — `binder1st`,
   8. — `binder2nd`,
   9. — `codecvt_mode`,
   10. — `codecvt_utf16`,
   11. — `codecvt_utf8`,
   12. — `codecvt_utf8_utf16`,
   13. — `const_mem_fun1_ref_t`,
   14. — `const_mem_fun1_t`,
   15. — `const_mem_fun_ref_t`,
   16. — `const_mem_fun_t`,
   17. — `consume_header`,
   18. — `generate_header`,
   19. — `get_temporary_buffer`,
   20. — `get_unexpected`,
   21. — `gets`,
   22. — `is_literal_type`,
   23. — `is_literal_type_v`,
   24. — `little_endian`,
   25. — `mem_fun1_ref_t`,
   26. — `mem_fun1_t`,
   27. — `mem_fun_ref_t`,
   28. — `mem_fun_ref`,
   29. — `mem_fun_t`,
   30. — `mem_fun`,
   31. — `not1`,
   32. — `not2`,
   33. — `pointer_to_binary_function`,
   34. — `pointer_to_unary_function`,
   35. — `ptr_fun`,
   36. — `random_shuffle`,
   37. — `raw_storage_iterator`,
   38. — `result_of`,
   39. — `result_of_t`,
   40. — `return_temporary_buffer`,
   41. — `set_unexpected`,
   42. — `unary_function`,
   43. — `unary_negate`,
   44. — `uncaught_exception`,
   45. — `unexpected`, and
   46. — `unexpected_handler`.
2. The following names are reserved as member types for previous standardization, and may not be used as a name for object-like macros in portable code:
   1. — `argument_type`,
   2. — `first_argument_type`,
   3. — `io_state`,
   4. — `open_mode`,
   5. — `second_argument_type`, and
   6. — `seek_dir`.
3. The name `stossc` is reserved as a member function for previous standardization, and may not be used as a name for function-like macros in portable code.

4. The header names <ccomplex>, <ciso646>, <codecvt>, <cstdalign>, <cstdbool>, and <ctgmath> are reserved for previous standardization.

D.20 Deprecated Standard code conversion facets [depr.locale.stdcvt]

1. The header <codecvt> provides code conversion facets for various character encodings.

D.20.1 Header <codecvt> synopsis [depr.codecvt.syn]

```
namespace std {
  enum codecvt_mode {
    consume_header = 4,
    generate_header = 2,
    little_endian = 1
  };

  template<class Elem, unsigned long Maxcode = 0x10ffff,
    codecvt_mode Mode = (codecvt_mode)0>
  class codecvt_utf8
    : public codecvt<Elem, char, mbstate_t> {
  public:
    explicit codecvt_utf8(size_t refs = 0);
    ~codecvt_utf8();
  };

  template<class Elem, unsigned long Maxcode = 0x10ffff,
    codecvt_mode Mode = (codecvt_mode)0>
  class codecvt_utf16
    : public codecvt<Elem, char, mbstate_t> {
  public:
    explicit codecvt_utf16(size_t refs = 0);
    ~codecvt_utf16();
  };

  template<class Elem, unsigned long Maxcode = 0x10ffff,
    codecvt_mode Mode = (codecvt_mode)0>
  class codecvt_utf8_utf16
    : public codecvt<Elem, char, mbstate_t> {
  public:
    explicit codecvt_utf8_utf16(size_t refs = 0);
    ~codecvt_utf8_utf16();
  };
}
```

D.20.2 Requirements [depr.locale.stdcvt.req]

1. For each of the three code conversion facets codecvt_utf8, codecvt_utf16, and codecvt_utf8_utf16:
   1. — Elem is the wide-character type, such as wchar_t, char16_t, or char32_t.
   2. — Maxcode is the largest wide-character code that the facet will read or write without reporting a conversion error.
   3. — If (Mode & consume_header), the facet shall consume an initial header sequence, if present, when reading a multibyte sequence to determine the endianness of the subsequent multibyte sequence to be read.
   4. — If (Mode & generate_header), the facet shall generate an initial header sequence when writing a multibyte sequence to advertise the endianness of the subsequent multibyte sequence to be written.
   5. — If (Mode & little_endian), the facet shall generate a multibyte sequence in little-endian order, as opposed to the default big-endian order.
2. For the facet codecvt_utf8:
   1. — The facet shall convert between UTF-8 multibyte sequences and UCS2 or UTF-32 (depending on the size of Elem) within the program.
   2. — Endianness shall not affect how multibyte sequences are read or written.
   3. — The multibyte sequences may be written as either a text or a binary file.
3. For the facet codecvt_utf16:
   1. — The facet shall convert between UTF-16 multibyte sequences and UCS2 or UTF-32 (depending on the size of Elem) within the program.
   2. — Multibyte sequences shall be read or written according to the Mode flag, as set out above.
   3. — The multibyte sequences may be written only as a binary file. Attempting to write to a text file produces undefined behavior.
4. For the facet codecvt_utf8_utf16:
   1. — The facet shall convert between UTF-8 multibyte sequences and UTF-16 (one or two 16-bit codes) within the program.
   2. — Endianness shall not affect how multibyte sequences are read or written.
   3. — The multibyte sequences may be written as either a text or a binary file.
5. The encoding forms UTF-8, UTF-16, and UTF-32 are specified in ISO/IEC 10646. The encoding form UCS-2 is specified in ISO/IEC 10646-1:1993.

D.21.1 Class template wstring_convert [depr.conversions.string]

1. Class template `wstring_convert` performs conversions between a wide string and a byte string. It lets you specify a code conversion facet (like class template `codecvt`) to perform the conversions, without affecting any streams or locales. [*Example:* If you ~~want to use the code conversion facet `codecvt_utf8`~~ have a code conversion facet called `codecvt_utf8` that you want to use to output to `cout` a UTF-8 multibyte sequence corresponding to a wide string, but you don't want to alter the locale for `cout`, you can write something like:

```
std::wstring_convert<std::codecvt_utf8<wchar_t>> myconv;
std::string mbstring = myconv.to_bytes(L"Hello\n");
std::cout << mbstring;
```

— *end example*]

Draft compatibility note for Annex C.

Weak recommendation: take no action at this time.

No change to draft.

LEWGI Review: To be determined...

D.21 Deprecated convenience conversions [depr.conversions]

First deprecated: C++17

This feature was originally proposed for C++11 by paper N2401 and deprecated for C++17 by paper P0618R0. As noted at the time, the feature was underspecified and would require more work than we wished to invest to bring it up to standard. Since then SG16 has been convened and is producing a steady stream of work to bring reliable well-specified Unicode support to C++.

Given vendors propensity to provide ongoing support for these names under the zombie name reservations, the strong recommendation of this paper is to remove this library immediately from the C++23 standard. The weak recommendation is to do the minimal work to clean up the wording to use the more precise terms that replaced *Requires* clauses, waiting until SG16 (or some other entity) produces a clean replacement for this facility for users to migrate to before removal.

Strong recommendation: Remove this facility from the standard at the earliest opportunity.

16.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization:
    1. — `auto_ptr`,
    2. — `auto_ptr_ref`,
    3. — `binary_function`,
    4. — `binary_negate`,
    5. — `bind1st`,
    6. — `bind2nd`,
    7. — `binder1st`,
    8. — `binder2nd`,
    9. — `const_mem_fun1_ref_t`,
    10. — `const_mem_fun1_t`,
    11. — `const_mem_fun_ref_t`,
    12. — `const_mem_fun_t`,
    13. — `get_temporary_buffer`,
    14. — `get_unexpected`,
    15. — `gets`,
    16. — `is_literal_type`,
    17. — `is_literal_type_v`,
    18. — `mem_fun1_ref_t`,
    19. — `mem_fun1_t`,
    20. — `mem_fun_ref_t`,
    21. — `mem_fun_ref`,
    22. — `mem_fun_t`,
    23. — `mem_fun`,
    24. — `not1`,
    25. — `not2`,
    26. — `pointer_to_binary_function`,
    27. — `pointer_to_unary_function`,

28. — `ptr_fun`,
29. — `random_shuffle`,
30. — `raw_storage_iterator`,
31. — `result_of`,
32. — `result_of_t`,
33. — `return_temporary_buffer`,
34. — `set_unexpected`,
35. — `unary_function`,
36. — `unary_negate`,
37. — `uncaught_exception`,
38. — `unexpected`, ~~and~~
39. — `unexpected_handler`~~.~~,
40. — `wbuffer_convert`, and
41. — `wstring_convert`.

2. The following names are reserved as member types for previous standardization, and may not be used as a name for object-like macros in portable code:
   1. — `argument_type`,
   2. — `first_argument_type`,
   3. — `io_state`,
   4. — `open_mode`,
   5. — `second_argument_type`, and
   6. — `seek_dir`.
3. ~~The name `stossc` is reserved as a member function for previous standardization, and may not be used as a name for function-like macros in portable code.~~ The following names are reserved as member functions for previous standardization, and may not be used as a name for function-like macros in portable code:
   1. — `converted`,
   2. — `from_bytes`,
   3. — `stossc`, and
   4. — `to_bytes`.
4. The header names `<ccomplex>`, `<ciso646>`, `<cstdalign>`, `<cstdbool>`, and `<ctgmath>` are reserved for previous standardization.

~~D.21 Deprecated convenience conversion interfaces [depr.conversions]~~

1. ~~The header `<locale>` (28.2) has the following additions:~~

```
namespace std {
  template<class Codecvt, class Elem = wchar_t,
           class WideAlloc = allocator<Elem>,
           class ByteAlloc = allocator<char>>
    class wstring_convert;
  template<class Codecvt, class Elem = wchar_t,
           class Tr = char_traits<Elem>>
    class wbuffer_convert;
}
```

~~D.21.1 Class template wstring_convert [depr.conversions.string]~~

1. ~~Class template `wstring_convert` performs conversions between a wide string and a byte string. It lets you specify a code conversion facet (like class template `codecvt`) to perform the conversions, without affecting any streams or locales. [*Example:* If you want to use the code conversion facet `codecvt_utf8` to output to `cout` a UTF-8 multibyte sequence corresponding to a wide string, but you don't want to alter the locale for `cout`, you can write something like:~~

```
wstring_convert<std::codecvt_utf8<wchar_t>> myconv;
std::string mbstring = myconv.to_bytes(L"Hello\n");
std::cout << mbstring;
```

~~— *end example*]~~

```
namespace std {
template <class Codecvt, class Elem = wchar_t,
          class WideAlloc = allocator<Elem>,
          class ByteAlloc = allocator<char>>
  class wstring_convert {
  public:
    using byte_string = basic_string<char, char_traits<char>, ByteAlloc>;
    using wide_string = basic_string<Elem, char_traits<Elem>, WideAlloc>;
    using state_type  = typename Codecvt::state_type;
    using int_type    = typename wide_string::traits_type::int_type;

    wstring_convert() : wstring_convert(new Codecvt) {}
```

```
    explicit wstring_convert(Codecvt* pcvt = new Codecvt);
    wstring_convert(Codecvt* pcvt, state_type state);
    explicit wstring_convert(const byte_string& byte_err,
                             const wide_string& wide_err = wide_string());
    ~wstring_convert();

    wstring_convert(const wstring_convert&) = delete;
    wstring_convert& operator=(const wstring_convert&) = delete;

    wide_string from_bytes(char byte);
    wide_string from_bytes(const char* ptr);
    wide_string from_bytes(const byte_string& str);
    wide_string from_bytes(const char* first, const char* last);

    byte_string to_bytes(Elem wchar);
    byte_string to_bytes(const Elem* wptr);
    byte_string to_bytes(const wide_string& wstr);
    byte_string to_bytes(const Elem* first, const Elem* last);

    size_t converted() const noexcept;
    state_type state() const;

  private:
    byte_string byte_err_string;  // Exposition only
    wide_string wide_err_string;  // Exposition only
    Codecvt* cvtptr;              // Exposition only
    state_type cvtstate;          // Exposition only
    size_t cvtcount;              // Exposition only
  };
}
```

2. The class template describes an object that controls conversions between wide string objects of class `basic_string<Elem, char_traits<Elem>, WideAlloc>` and byte string objects of class `basic_string<char, char_traits<char>, ByteAlloc>`. The class template defines the types `wide_string` and `byte_string` as synonyms for these two types. Conversion between a sequence of `Elem` values (stored in a `wide_string` object) and multibyte sequences (stored in a `byte_string` object) is performed by an object of class `Codecvt`, which meets the requirements of the standard code-conversion facet `codecvt<Elem, char, mbstate_t>`.
3. An object of this class template stores:
   1. — `byte_err_string` — a byte string to display on errors
   2. — `wide_err_string` — a wide string to display on errors
   3. — `cvtptr` — a pointer to the allocated conversion object (which is freed when the `wstring_convert` object is destroyed)
   4. — `cvtstate` — a conversion state object
   5. — `cvtcount` — a conversion count

```
using byte_string = basic_string<char, char_traits<char>, ByteAlloc>;
```

4. The type shall be a synonym for `basic_string<char, char_traits<char>, ByteAlloc>`

```
size_t converted() const noexcept;
```

5. *Returns:* `cvtcount`.

```
wide_string from_bytes(char byte);
wide_string from_bytes(const char* ptr);
wide_string from_bytes(const byte_string& str);
wide_string from_bytes(const char* first, const char* last);
```

6. *Effects:* The first member function shall convert the single-element sequence `byte` to a wide string. The second member function shall convert the null-terminated sequence beginning at `ptr` to a wide string. The third member function shall convert the sequence stored in `str` to a wide string. The fourth member function shall convert the sequence defined by the range `[first, last)` to a wide string.
7. In all cases:
   1. — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.
   2. — The number of input elements successfully converted shall be stored in `cvtcount`.
8. *Returns:* If no conversion error occurs, the member function shall return the converted wide string. Otherwise, if the object was constructed with a wide-error string, the member function shall return the wide-error string. Otherwise, the member function throws an object of class `range_error`.

```
using int_type = typename wide_string::traits_type::int_type;
```

9. The type shall be a synonym for `wide_string::traits_type::int_type`.

```
state_type state() const;
```

10. returns cvtstate.

```
using state_type = typename Codecvt::state_type;
```

11. The type shall be a synonym for Codecvt::state_type.

```
byte_string to_bytes(Elem wchar);
byte_string to_bytes(const Elem* wptr);
byte_string to_bytes(const wide_string& wstr);
byte_string to_bytes(const Elem* first, const Elem* last);
```

12. *Effects:* The first member function shall convert the single-element sequence wchar to a byte string. The second member function shall convert the null-terminated sequence beginning at wptr to a byte string. The third member function shall convert the sequence stored in wstr to a byte string. The fourth member function shall convert the sequence defined by the range [first, last) to a byte string.
13. In all cases:
    1. — If the cvtstate object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.
    2. — The number of input elements successfully converted shall be stored in cvtcount.
14. *Returns:* If no conversion error occurs, the member function shall return the converted byte string. Otherwise, if the object was constructed with a byte-error string, the member function shall return the byte-error string. Otherwise, the member function shall throw an object of class range_error.

```
using wide_string = basic_string<Elem, char_traits<Elem>, WideAlloc>;
```

15. The type shall be a synonym for basic_string<Elem, char_traits<Elem>, WideAlloc>.

```
explicit wstring_convert(Codecvt* pcvt = new Codecvt);
wstring_convert(Codecvt* pcvt, state_type state);
explicit wstring_convert(const byte_string& byte_err,
    const wide_string& wide_err = wide_string());
```

16. *Requires:* For the first and second constructors, pcvt != nullptr.
17. *Effects:* The first constructor shall store pcvt in cvtptr and default values in cvtstate, byte_err_string, and wide_err_string. The second constructor shall store pcvt in cvtptr, state in cvtstate, and default values in byte_err_string and wide_err_string; moreover the stored state shall be retained between calls to from_bytes and to_bytes. The third constructor shall store new Codecvt in cvtptr, state_type() in cvtstate, byte_err in byte_err_string, and wide_err in wide_err_string.

```
~wstring_convert();
```

18. *Effects:* The destructor shall delete cvtptr.

D.21.2 Class template wbuffer_convert [depr.conversions.buffer]

1. Class template wbuffer_convert looks like a wide stream buffer, but performs all its I/O through an underlying byte stream buffer that you specify when you construct it. Like class template wstring_convert, it lets you specify a code conversion facet to perform the conversions, without affecting any streams or locales.

```
namespace std {
template <class Codecvt, class Elem = wchar_t, class Tr = char_traits<Elem>>
  class wbuffer_convert
    : public basic_streambuf<Elem, Tr> {
  public:
    using state_type = typename Codecvt::state_type;

    wbuffer_convert() : wbuffer_convert(nullptr) {}
    explicit wbuffer_convert(streambuf* bytebuf = 0,
                             Codecvt* pcvt = new Codecvt,
                             state_type state = state_type());

    ~wbuffer_convert();

    wbuffer_convert(const wbuffer_convert&) = delete;
    wbuffer_convert& operator=(const wbuffer_convert&) = delete;

    streambuf* rdbuf() const;
    streambuf* rdbuf(streambuf* bytebuf);
```

```
    state_type state() const;

  private:
    streambuf* bufptr;          // exposition only
    Codecvt* cvtptr;            // exposition only
    state_type cvtstate;        // exposition only
  };
}
```

2. The class template describes a stream buffer that controls the transmission of elements of type `Elem`, whose character traits are described by the class `Tr`, to and from a byte stream buffer of type `streambuf`. Conversion between a sequence of `Elem` values and multibyte sequences is performed by an object of class `Codecvt`, which shall meet the requirements of the standard code-conversion facet `codecvt<Elem, char, mbstate_t>`.
3. An object of this class template stores:
   1. `bufptr` — a pointer to its underlying byte stream buffer
   2. `cvtptr` — a pointer to the allocated conversion object (which is freed when the `wbuffer_convert` object is destroyed)
   3. `cvtstate` — a conversion state object

```
state_type state() const;
```

4. *Returns:* `cvtstate`.

```
streambuf* rdbuf() const;
```

5. *Returns:* `bufptr`.

```
streambuf* rdbuf(streambuf* bytebuf);
```

6. *Effects:* Stores `bytebuf` in `bufptr`.
7. *Returns:* The previous value of `bufptr`.

```
using state_type = typename Codecvt::state_type;
```

8. The type shall be a synonym for `Codecvt::state_type`.

```
explicit wbuffer_convert(streambuf* bytebuf = 0,
    Codecvt* pcvt = new Codecvt, state_type state = state_type());
```

9. *Requires:* `pcvt != nullptr`.
10. *Effects:* The constructor constructs a stream buffer object, initializes `bufptr` to `bytebuf`, initializes `cvtptr` to `pcvt`, and initializes `cvtstate` to `state`.

```
~wbuffer_convert();
```

11. *Effects:* The destructor shall delete `cvtptr`.

Draft compatibility note for Annex C.

Weak recommendation: Clean up wording.

Update wording to replace *Requires* clauses (see D.10 Requires paragraph).

LEWGI Review: To be determined...

D.22 Deprecated locale category facets [depr.locale.category]

First deprecated: C++20

This feature was added as part of the initial basic support for Unicode types in C++11 by paper N2238 and deprecated on the recommendation of SG16 for C++20 by paper P0482R6.

As SG16 do not report any urgent issue relating to this deprecated feature, and are still working through the process of providing clean Unicode support in the C++ standard library, and given the deprecation is as recent as C++20, both the strong and weak recommendations are to take no action on this feature at this time.

Strong recommendation: Take no action.

No change to draft.

Weak recommendation: Take no action.

No change to draft.

LEWGI Review: To be determined...

D.23 Deprecated filesystem path factory functions [depr.fs.path.factory]

First deprecated: C++20

A factory function to create path names from UTF-8 sequences was part of the original filesystem library adopted for C++17. However, this was the only string-based factory function, as the preferred interface is to simply construct a path with a string of the corresponding type/encoding. This factory function was deprecated in C++20 with the addition of `char8_t` and the ability to now invoke a specific constructor for UTF-8 encoded (and typed) strings. See P0482R6 for details.

The legacy API continues to function, but is more cumbersome than necessary. There appears to be no compelling case that the API is a risk through misuse. Therefore, given it was so recently deprecated, the strong recommendation is to retain this feature in Annex D for C++23, giving the community time to catch up, and consider removal again for C++26. However, the current wording does not follow library best practices, and should be updated to better specify the *Requires* clauses.

However, while it does no active harm, there is always a cost to maintaining text in the standard. The application of zombie names means that even if we remove this clause from Annex D in C++23, standard library vendors are likely to continue shipping to meet customer demand for some time to come. So the weak recommendation is to add the names to the zombie clause, and remove immediately from C++23.

Strong recommendation: Clean up wording.

Update wording to replace *Requires* clauses (see D.10 Requires paragraph).

Weak recommendation: Remove this feature from C++23

16.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization:
    1. — `auto_ptr`,
    2. — `auto_ptr_ref`,
    3. — `binary_function`,
    4. — `binary_negate`,
    5. — `bind1st`,
    6. — `bind2nd`,
    7. — `binder1st`,
    8. — `binder2nd`,
    9. — `const_mem_fun1_ref_t`,
    10. — `const_mem_fun1_t`,
    11. — `const_mem_fun_ref_t`,
    12. — `const_mem_fun_t`,
    13. — `get_temporary_buffer`,
    14. — `get_unexpected`,
    15. — `gets`,
    16. — `is_literal_type`,
    17. — `is_literal_type_v`,
    18. — `mem_fun1_ref_t`,
    19. — `mem_fun1_t`,
    20. — `mem_fun_ref_t`,
    21. — `mem_fun_ref`,
    22. — `mem_fun_t`,
    23. — `mem_fun`,
    24. — `not1`,
    25. — `not2`,
    26. — `pointer_to_binary_function`,
    27. — `pointer_to_unary_function`,
    28. — `ptr_fun`,
    29. — `random_shuffle`,
    30. — `raw_storage_iterator`,
    31. — `result_of`,
    32. — `result_of_t`,

33. — `return_temporary_buffer`,
34. — `set_unexpected`,
35. — `u8path`,
36. — `unary_function`,
37. — `unary_negate`,
38. — `uncaught_exception`,
39. — `unexpected`, and
40. — `unexpected_handler`.

D.23 Deprecated filesystem path factory functions [depr.fs.path.factory]

```
template<class Source>
  path u8path(const Source& source);
template<class InputIterator>
  path u8path(InputIterator first, InputIterator last);
```

1. *Requires:* The `source` and `[first, last)` sequences are UTF-8 encoded. The value type of `Source` and `InputIterator` is `char` or `char8_t`. `Source` meets the requirements specified in 29.11.7.3.
2. *Returns:*
   1. — If `value_type` is `char` and the current native narrow encoding (29.11.7.2.2) is UTF-8, return `path(source)` or `path(first, last)`; otherwise,
   2. — if `value_type` is `wchar_t` and the native wide encoding is UTF-16, or if `value_type` is `char16_t` or `char32_t`, convert `source` or `[first, last)` to a temporary, `tmp`, of type `string_type` and return `path(tmp)`; otherwise,
   3. — convert `source` or `[first, last)` to a temporary, `tmp`, of type `u32string` and return `path(tmp)`.
3. *Remarks:* Argument format conversion (29.11.7.2.1) applies to the arguments for these functions. How Unicode encoding conversions are performed is unspecified.
4. [*Example:* A string is to be read from a database that is encoded in UTF-8, and used to create a directory using the native encoding for filenames:

   ```
   namespace fs = std::filesystem;
   std::string utf8_string = read_utf8_data();
   fs::create_directory(fs::u8path(utf8_string));
   ```

   For POSIX-based operating systems with the native narrow encoding set to UTF-8, no encoding or type conversion occurs.

   For POSIX-based operating systems with the native narrow encoding not set to UTF-8, a conversion to UTF-32 occurs, followed by a conversion to the current native narrow encoding. Some Unicode characters may have no native character set representation.

   For Windows-based operating systems a conversion from UTF-8 to UTF-16 occurs. — *end example*] [*Note:* The example above is representative of a historical use of `filesystem::u8path`. Passing a `std::u8string` to `path`'s constructor is preferred for an indication of UTF-8 encoding more consistent with `path`'s handling of other encodings. — *end note*]

Draft compatibility note for Annex C.

LEWGI Review: To be determined...

D.24 Deprecated atomic operations [depr.atomics]

First deprecated: C++20

The original API to initialize atomic variables from C++11 was deprecated for C++20 when the `atomic` template was given a default constructor to do the right thing. See P0883R2 for details.

The legacy API continues to function, but is more cumbersome than necessary. There appears to be no compelling case that the API is a risk through misuse. Therefore, given it was so recently deprecated, the strong recommendation is to retain this feature in Annex D for C++23, giving the community time to catch up, and consider removal again for C++26.

However, while it does no active harm, there is always a cost to maintaining text in the standard. The application of zombie names means that even if we remove this clause from Annex D in C++23, standard library vendors are likely to continue shipping to meet customer demand for some time to come. So the weak recommendation is to add the names to the zombie clause, and remove immediately from C++23.

Additionally, the `volatile` qualified member functions of the `atomic` class template were deprecated for C++20 by paper P1831R1. Both the strong and weak recommendations are to leave this feature alone until the interaction with the non-deprecated non-member functions in the `<atomic>` header that take pointer-to-volatile-qualified type. Possible directions would be to deprecate those non-member functions too, or to undeprecate the volatile-qualified member functions.

Strong recommendation: Take no action. Reconsider for C++26.

No change to draft.

Weak recommendation: Remove deprecated atomic initialization feature from C++23

16.5.4.3.1 Zombie names [zombie.names]

1. In namespace `std`, the following names are reserved for previous standardization:
   1. — `ATOMIC_FLAG_INIT`,
   2. — `atomic_init`,
   3. — `ATOMIC_VAR_INIT`,
   4. — `auto_ptr`,
   5. — `auto_ptr_ref`,
   6. — `binary_function`,
   7. — `binary_negate`,
   8. — `bind1st`,
   9. — `bind2nd`,
   10. — `binder1st`,
   11. — `binder2nd`,
   12. — `const_mem_fun1_ref_t`,
   13. — `const_mem_fun1_t`,
   14. — `const_mem_fun_ref_t`,
   15. — `const_mem_fun_t`,
   16. — `get_temporary_buffer`,
   17. — `get_unexpected`,
   18. — `gets`,
   19. — `is_literal_type`,
   20. — `is_literal_type_v`,
   21. — `mem_fun1_ref_t`,
   22. — `mem_fun1_t`,
   23. — `mem_fun_ref_t`,
   24. — `mem_fun_ref`,
   25. — `mem_fun_t`,
   26. — `mem_fun`,
   27. — `not1`,
   28. — `not2`,
   29. — `pointer_to_binary_function`,
   30. — `pointer_to_unary_function`,
   31. — `ptr_fun`,
   32. — `random_shuffle`,
   33. — `raw_storage_iterator`,
   34. — `result_of`,
   35. — `result_of_t`,
   36. — `return_temporary_buffer`,
   37. — `set_unexpected`,
   38. — `unary_function`,
   39. — `unary_negate`,
   40. — `uncaught_exception`,
   41. — `unexpected`, and
   42. — `unexpected_handler`.

D.24 Deprecated atomic initialization [depr.atomics]

1. ~~The header <atomics> (31.2) has the following additions.~~

```
namespace std {
  template<class T>
    void atomic_init(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
    void atomic_init(atomic<T>*, typename atomic<T>::value_type) noexcept;

#define ATOMIC_VAR_INIT(value) see below

#define ATOMIC_FLAG_INIT see below
}
```

D.24.1 Volatile access [depr.atomics.volatile]

If an atomic specialization has one of the following overloads, then that overload participates in overload resolution even if `atomic<T>::is_always_lock_free` is `false`:

```
void store(T desired, memory_order order = memory_order::seq_cst) volatile noexcept;
T operator=(T desired) volatile noexcept;
T load(memory_order order = memory_order::seq_cst) const volatile noexcept;
operator T() const volatile noexcept;
T exchange(T desired, memory_order order = memory_order::seq_cst) volatile noexcept;
bool compare_exchange_weak(T& expected, T desired,
                           memory_order success, memory_order failure) volatile noexcept;
bool compare_exchange_strong(T& expected, T desired,
                             memory_order success, memory_order failure) volatile noexcept;
bool compare_exchange_weak(T& expected, T desired,
                           memory_order order = memory_order::seq_cst) volatile noexcept;
bool compare_exchange_strong(T& expected, T desired,
                             memory_order order = memory_order::seq_cst) volatile noexcept;
T fetch_key(T operand, memory_order order = memory_order::seq_cst) volatile noexcept;
T operator op=(T operand) volatile noexcept;
T* fetch_key(ptrdiff_t operand, memory_order order = memory_order::seq_cst) volatile noexcept;
```

~~**D.24.2 Non-member functions [depr.atomics.nonmembers]**~~

~~`template<class T>`~~
~~`  void atomic_init(volatile atomic<T>* object, typename atomic<T>::value_type desired) noexcept;`~~
~~`template<class T>`~~
~~`  void atomic_init(atomic<T>* object, typename atomic<T>::value_type desired) noexcept;`~~

1. ~~*Effects:* Equivalent to: `atomic_store_explicit(object, desired, memory_order::relaxed);`~~

~~**D.24.3 Operations on atomic types [depr.atomics.types.operations]**~~

~~`#define ATOMIC_VAR_INIT(value)` *see below*~~

1. ~~The macro expands to a token sequence suitable for constant initialization of an atomic variable of static storage duration of a type that is initialization-compatible with `value`. [*Note:* This operation may need to initialize locks. — *end note*] Concurrent access to the variable being initialized, even via an atomic operation, constitutes a data race. [*Example:*~~

   ~~`atomic<int> v = ATOMIC_VAR_INIT(5);`~~

   ~~— *end example*]~~

~~**D.24.4 Flag type and operations [depr.atomics.flag]**~~

~~`#define ATOMIC_FLAG_INIT see below`~~

1. ~~*Remarks:* The macro `ATOMIC_FLAG_INIT` is defined in such a way that it can be used to initialize an object of type `atomic_flag` to the clear state. The macro can be used in the form:~~

   ~~`atomic_flag guard = ATOMIC_FLAG_INIT;`~~

   ~~It is unspecified whether the macro can be used in other initialization contexts. For a complete static-duration object, that initialization shall be static.~~

Draft compatibility note for Annex C.

LEWGI Review: To be determined...

4 Other Directions

While practicing good housekeeping and clearing out Annex D for each release may be the preferred option, there are other approaches that may be taken.

Do Nothing

One approach, epitomized in the Java language, is that deprecated features are discouraged for future use, but guaranteed to remain available forever, and just accumulate.

This approach is rejected by this paper for a number of reasons. First, C++ has been relatively successful in actually removing its deprecated features in the past, a tradition we would like to continue. It also undercuts the available-forever rationale, as it is not a guarantee we have given

before.

A second concern is that we do not want to pay a cost to maintain deprecated components forever - restricting growth of the language for compatibility with deprecated features, or having to review the whole of Annex D and upgrade components for every new language release, in order to keep up with subtle shifts in the core language.

Undeprecate

If something is deprecated, but later (re)discovered to have value, then it could be revitalized and restored to the main standard. For example, this is exactly what happened to static function declarations when the unnamed namespace was given internal linkage - it is merely the classical way to say the same thing, and often clearer to write.

This may be a consideration for long-term deprecated features that don't appear to be going anywhere, such as the `strstream` facility, or the C headers. It may be appropriate to find them a home in the regular standard, and this is called out in the specific reviews of each facility above.

5 Integrated Proposed Wording

This section integrates the proposed wording of the above recommendations as the various working groups approve them for review by the Core and Library Working Groups. It will apply the wording for mandating Annex D if the recommendation is to retain a facility.

Collected summary of recommendations:

Subclause	Feature	Adopted Recommendation	Action
D.1	Arithmetic conversion on enumerations		
D.2	Implicit capture of `*this` by reference		
D.3	Comma operator in subscript expressions		
D.4	Array comparisons		
D.5	Deprecated use of `volatile`		
D.6	Reclare `constexpr` members		
D.7	Non-local use of TU-local entities		
D.8	Implicit special members		
D.9	C `<*.h>` headers		
D.10	*Requires:* clauses		
D.11	`relops`		
D.12	`char *` streams		
D.13	Deprecated type traits		
D.14	volatile `tuple` traits		
D.15	volatile `variant` traits		
D.16	`std::iterator`		
D.17	`move_iterator::operator->`		
D.18	C API to use `shared_ptr` atomically		
D.19	`basic_string::reserve()`		
D.20	`<codecvt>`		
D.21	`wstring_convert` et al.		
D.22	Deprecated locale category facets		
D.23	`filesystem::u8path`		
D.24	atomic operations		

Full wording follows:

> Wording will accumulate (L)EWG recommendations when approved.

7.4 Usual arithmetic conversions [expr.arith.conv]

1. Many binary operators that expect operands of arithmetic or enumeration type cause conversions and yield result types in a similar way. The purpose is to yield a common type, which is also the type of the result. This pattern is called the *usual arithmetic conversions*, which are defined as follows:
    1. — If either operand is of scoped enumeration type (9.7.1), no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.
    2. — Otherwise, if either operand is of unscoped enumeration type (9.7.1) and the other operand is of a floating-point type, the expression is ill-formed.

3. — Otherwise, if either operand is of type `long double`, the other shall be converted to `long double`.
4. — Otherwise, if either operand is `double`, the other shall be converted to `double`.
5. — Otherwise, if either operand is `float`, the other shall be converted to `float`.
6. — Otherwise, the integral promotions (7.3.6) shall be performed on both operands.[56] Then the following rules shall be applied to the promoted operands:
    1. — If both operands have the same type, no further conversion is needed.
    2. — Otherwise, if both operands have signed integer types or both have unsigned integer types, the operand with the type of lesser integer conversion rank shall be converted to the type of the operand with greater rank.
    3. — Otherwise, if the operand that has unsigned integer type has rank greater than or equal to the rank of the type of the other operand, the operand with signed integer type shall be converted to the type of the operand with unsigned integer type.
    4. — Otherwise, if the type of the operand with signed integer type can represent all of the values of the type of the operand with unsigned integer type, the operand with unsigned integer type shall be converted to the type of the operand with signed integer type.
    5. — Otherwise, both operands shall be converted to the unsigned integer type corresponding to the type of the operand with signed integer type.

2. If ~~one~~either operand is of unscoped enumeration type and the other operand is of a different enumeration type ~~or a floating-point type~~, this behavior is deprecated (D.1).

7.6.1.1 Subscripting [expr.sub]

2. ~~[*Note:*~~ A comma expression (7.6.20) appearing as the *expr-or-braced-init-list* of a subscripting expression is ill-formed.~~deprecated; see D.3. — *end note*]~~

7.6.9 Relational operators [expr.rel]

1. The relational operators group left-to-right. [*Example:* `a<b<c` means `(a<b)<c` and not `(a<b)&&(b<c)`. — *end example*]

```
relational-expression : compare-expression
relational-expression < compare-expression
relational-expression > compare-expression
relational-expression <= compare-expression
relational-expression >= compare-expression
```

   The lvalue-to-rvalue (7.3.1)~~, array-to-pointer (7.3.2),~~ and function-to-pointer(7.3.3) standard conversions are performed on the operands. If at least one of the operands is a pointer, array-to-pointer conversions (7.3.2) are performed. ~~The comparison is deprecated if both operands were of array type prior to these conversions (D.4).~~
2. The converted operands shall have arithmetic, enumeration, or pointer type. The operators < (less than), > (greater than), <= (less than or equal to), and >= (greater than or equal to) all yield `false` or `true`. The type of the result is `bool`.
3. The usual arithmetic conversions (7.4) are performed on operands of arithmetic or enumeration type. If both operands are pointers, pointer conversions (7.3.11) and qualification conversions (7.3.5) are performed to bring them to their composite pointer type (7.2.2). After conversions, the operands shall have the same type.
4. The result of comparing unequal pointers to objects ...

7.6.10 Equality operators [expr.eq]

```
equality-expression :
    relational-expression
    equality-expression == relational-expression
    equality-expression != relational-expression
```

1. The == (equal to) and the `!=` (not equal to) operators group left-to-right. The lvalue-to-rvalue (7.3.1)~~, array-to-pointer (7.3.2),~~ and function-to-pointer (7.3.3) standard conversions are performed on the operands. ~~The comparison is deprecated if both operands were of array type prior to these conversions (D.4).~~
2. If at least one of the operands is a pointer, array-to-pointer conversions (7.3.2), pointer conversions (7.3.11), function pointer conversions (7.3.13), and qualification conversions (7.3.5) are performed on both operands to bring them to their composite pointer type (7.2.2). ~~Comparing pointers is defined as follows:~~
3. The converted operands shall have arithmetic, enumeration, pointer, or pointer-to-member type, or type `std::nullptr_t`. The operators == and `!=` both yield `true` or `false`, i.e., a result of type `bool`. In each case below, the operands shall have the same type after the specified conversions have been applied.
4. Comparing pointers is defined as follows:
    1. — If one pointer represents the address of a complete object, and another pointer represents the address one past the last element of a different complete object,[79] the result of the comparison is unspecified.
    2. — Otherwise, if the pointers are both null, both point to the same function, or both represent the same address (6.8.2), they compare equal.
    3. — Otherwise, the pointers compare unequal.

5. If at least one of the operands is a pointer to member, ...

7.6.20 Comma operator [expr.comma]

3. [*Note:* A comma expression (7.6.20) appearing as the *expr-or-braced-init-list* of a subscripting expression is ill-formed~~deprecated; see D.3~~. *—end note*]

C.1 C++ and ISO C++ 2020

1. This subclause lists the differences between C++ and ISO C++ 2020 (ISO/IEC 14882:2020, Programming Languages — C++), by the chapters of this document.

C.1.X Clause 7: Expressions [diff.cpp20.expr]

Affected subclause: 7.4 *Note for editors: [expr.arith.conv]*

Change: Unscoped enumerations do not implicitly promote to integral type in expressions with floating point types.

Rationale: ...

Effect on original feature: A valid C++ 2020 involving both an unscoped enumeration and a floating point value will be rejected as ill-formed in this International Standard. Either argument could be explicitly conerted with a cast, or the enumeration could be explicitly promoted to an integer with unary operator +, for no change of meaning since C++ 2020. [*Example:*

```
enum multipliers { gigaseconds = 1'000'000 };
double century_ish = 3.14 *  gigaseconds;  // ill-formed; previously well-formed
double century_est = 3.14 * +gigaseconds;  // OK
```

—end example]

Affected subclause: 7.6.1.1 *Note for editors: [expr.sub]*

Change: Cannot parse unparenthesized comma expressions as subscript expressions.

Rationale: The old behavior was surprising for users familiar with other languages. It was removed to allow a future extension to support overloading the subscript operator for multiple arguments.

Effect on original feature: A valid C++ 2020 program using a comma expression as the argument to a subscript expression will be rejected as ill-formed in this International Standard. The intended comma expression can be enclosed in parentheses for no change of meaning since C++ 2020. [*Example:*

```
void f(int *a, int b, int c) {
   a[b,c];          // ill-formed; previously well-formed
   a[(b,c)];        // OK
}
```

Add an example of a DSEL

—end example]

Affected subclause: 7.6.9 and 7.6.10 *Note for editors: [expr.rel] and [expr.eq]*

Change: Cannot compare two objects of array type.

Rationale: The old behavior was confusing, as it did not compare the contents of the two arrays, but compare their addresses. Depending on context, this would either report whether the two arrays were the same object, or have an unspecified result.

Effect on original feature: A valid C++ 2020 program directly comparing two array objects will be rejected as ill-formed in this International Standard. [*Example:*

```
int arr1[5];
int arr2[5];
bool same = arr1 ==  arr2;      // ill-formed; previously well-formed
bool idem = arr1 == +arr2;      // compare addresses, unspecified result
```

—end example]

C.5.3 Clause 7: expressions [diff.expr]

Affected subclause: 7.4 *Note for editors: [expr.arith.conv]*

Change: Enumerations cannot be used in expressions with floating point types.

Rationale: ...

Effect on original feature: Deletion of semantically well-defined feature.

Difficulty of converting: Could be automated. Violations will be diagnosed by the C++ translator. The fix is to add a cast, or explicitly promote the `enum` object to an integer with unary operator `+`. For example:

How widely used: Common.

Affected subclause: 7.6.1.1 *Note for editors: [expr.sub]*

Change: Cannot parse unparenthesized comma expressions as subscript expressions.

Rationale: The old behavior was surprising for users familiar with other languages. It was removed to allow a future extension to support overloading the subscript operator for multiple arguments.

Effect on original feature: Deletion of semantically well-defined feature.

Difficulty of converting: Could be automated. Violations will be diagnosed by the C++ translator. The fix is to add parentheses. For example:

```
void f(int *a, int b, int c) {
   a[b,c];          // ill-formed; previously well-formed
   a[(b,c)];        // OK
}
```

How widely used: Rare.

Affected subclause: 7.6.9 and 7.6.10 *Note for editors: [expr.rel] and [expr.eq]*

Change: Cannot compare two objects of array type.

Rationale: The old behavior was confusing, as it did not compare the contents of the two arrays, but compare their addresses. Depending on context, this would either report whether the two arrays were the same object, or have an unspecified result.

Effect on original feature: Deletion of feature with unspecified behavior.

Difficulty of converting: Violations will be diagnosed by the C++ translator.

How widely used: Rare. In the cases where the result is well defined, it is reporting whether both arguments are the same object, using the same name.

D.1 Arithmetic conversion on enumerations [depr.arith.conv.enum]

1. The ability to apply the usual arithmetic conversions (7.4) on operands where one is of unscoped enumeration type and the other is of a different enumeration type ~~or a floating-point type~~ is deprecated. [Note: Three-way comparisons (7.6.8) between such operands are ill-formed. —*end note*] [*Example:*

```
enum E1 { e };
enum E2 { f };
bool b = e <= 3.7;              // deprecated
int k = f - e;                  // deprecated
auto cmp = e <=> f;             // error
```

—*end example*]

~~**D.3 Comma operator in subscript expressions [depr.comma.subscript]**~~

1. ~~A comma expression (7.6.20) appearing as the *expr-or-braced-init-list* of a subscripting expression (7.6.1.1) is deprecated. [*Note:* A parenthesized comma expression is not deprecated. —*end note*] [*Example:*~~

```
void f(int *a, int b, int c) {
      a[b,c];                   // deprecated
      a[(b,c)];                 // OK
}
```

~~—*end example*]~~

~~**D.4 Array comparisons [depr.array.comp]**~~

1. ~~Equality and relational comparisons (7.6.10, 7.6.9) between two operands of array type are deprecated. [*Note:* Three-way comparisons (7.6.8) between such operands are ill-formed. — *end note*] [*Example:*~~

```
int arr1[5];
int arr2[5];
bool same = arr1 == arr2;        // deprecated, same as &arr1[0] == &arr2[0],
                                 // does not compare array contents
auto cmp = arr1 <=> arr2;        // error
```

~~— *end example*]~~

6 Acknowledgements

Special thanks to Hal Finkel for allowing a late update to track the latest IS wording in the mailing. Thanks to Stephan T. Lavavej for the early review, and catching too many spelling and grammar errors!

Thanks to everyone who worked on flagging these facilities for deprecation, let's take the next step!

7 References

- N2007 PROPOSED LIBRARY ADDITIONS FOR CODE CONVERSION, P.J. Plauger
- N2238 Minimal Unicode support for the standard library, Matthew Austern
- N2401 Code Conversion Facets for the Standard C++ Library, P.J. Plauger
- N3201 Moving right along, Bjarne Stroustrup
- N3203 Tightening the conditions for generating implicit moves, Jens Maurer
- N4086 Removing trigraphs??!, Richard Smith
- N4861 C++ Standard Working Draft (at inception of C++23), Richard Smith
- P0174R2 Deprecating Vestigial Library Parts in Ci++17, Alisdair Meredith
- P0386R2 Inline Variables, Hal Finkel and Richard Smith
- P0482R6 char8_t: A type for UTF-8 characters and strings, Tom Honermann
- P0618R0 Deprecating `<codecvt>`, Alisdair Meredith
- P0619R4 Reviewing Deprecated Facilities of C++17 for C++20, Alisdair Meredith, Stephan T. Lavavej, Tomasz Kamiński
- P0718R2 Revising `atomic_shared_ptr` for C++20, Alisdair Meredith
- P0767R1 Deprecate POD, Jens Maurer
- P0768R1 Library Support for the Spaceship (Comparison) Operator, Walter E. Brown
- pP788R3 Standard Library Specification in a Concepts and Contracts World, Walter E. Brown
- P0806R2 Deprecate implicit capture of `this` via `[=]`, Thomas Köppe
- P0883R2 Fixing Atomic Initialization, Nicolai Josuttis
- P0966R1 `string::reserve` Should Not Shrink, Mark Zeren, Andrew Luo
- P1152R4 Deprecating `volatile`, JF Bastien
- P1161R3 Deprecate uses of the comma operator in subscripting expressions, Corentin Jabot
- P1120R0 Consistency improvements for <=> and other comparison operators, Richard Smith
- P1252R2 Ranges Design Cleanup, Casey Carter
- P1815R2 Translation-unit-local entities, S. Davis Herring
- P1831R1 Deprecating `volatile`: library, JF Bastien
- P2128R1 Multidimensional subscript operator, Mark Hoemmen, DS Hollman, Corentin Jabot, Isabella Muerte, Christian Trott